

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Multidisciplinary Project

Project Report

Application of AIoT in Farming Management

Advisor: Dr. Pham Hoang Anh
Students: Pham Nguyen Minh Hieu - 2252215
Nguyen Ngoc Khoi - 2252378
Tran Xuan Bach - 2252058
Nguyen Doan Hai Bang - 2252078
Le Quang Dang - 2252152

HO CHI MINH CITY, JUNE 2025



Contents

1	Member List & Workload	4
2	Introduction	5
2.1	Smart farm	5
2.2	AIoT	5
3	User Requirements	6
3.1	Functional Requirements	6
3.1.1	Application aspects	6
3.1.2	Internet of Things (IoT) aspects	6
3.1.3	Artificial Intelligence (AI) aspects	6
3.2	Non-functional Requirements	7
4	System Requirements	8
4.1	Use-case Description	8
4.2	Use-case Diagram	14
5	System Device	15
5.1	Microcontroller & Edge Devices	15
5.2	Device Information	16
5.2.1	Input	17
5.2.2	Output	17
6	Gate Way/Edge Device	18
6.1	Command & Control Bridging	18
6.2	Telemetry Aggregation & Streaming	18
6.3	Local Alarm and Multimode Control	18
6.4	Gateway Operations	19
6.4.1	Background Task Finite-State Machines	19
7	MQTT server	22
7.1	MQTT Broker	22
7.2	ThingsBoard	22
7.3	CoreIoT	22
8	System Design	24
8.1	Behavior Diagram	24
8.2	Sequence Diagram	27
8.3	Class Diagram	31



9	Technology	37
9.1	Front-end	37
9.1.1	HTML	37
9.1.2	CSS	38
9.1.3	SCSS	38
9.1.4	JSX	39
9.2	Back-end	41
9.2.1	Python	41
9.2.2	REST API	41
9.2.3	JSON Backend	42
10	Incorporate CNNs for Prediction	44
10.1	Dataset Description	44
10.2	Basic Statistics and Machine Learning Concepts	45
11	Incorporate YOLO for Detection	47
11.1	Model Architecture	47
11.2	Dataset	47
11.2.1	Label Definitions	48
11.2.2	Exploratory Data Analysis	48
11.3	Activity Diagram	51
11.4	Sequence Diagram	52
11.5	Training	53
11.6	Evaluation	53
11.6.1	Result	53
11.6.2	Test set Performance	54
12	Design Pattern	56
12.1	Observer	56
12.2	Module	56
13	Main Module	58
13.0.1	Module 1: Receive and display data from devices	58
13.0.2	Module 2: Check received data exceeds allowed threshold	58
13.0.3	Module 3: Device control	59
13.1	Module 4: Record activities	59
13.1.1	Module 5: Web application	60
14	UI Description	61
14.1	Core Dashboard	61
14.1.1	System Objectives	61
14.1.2	Main Dashboard Overview	62
14.1.3	Real-Time Environmental Monitoring Subdashboard	66
14.1.4	CoreIOT Dashboard Summary	71



14.2	Project Web App	72
14.2.1	User Authentication: Login and Signup	73
14.2.2	Control Panel and Threshold Settings	74
14.2.3	Alerts and Receiving Notifications	75
14.2.4	Add Notes	76
14.2.5	Create Reminder	77
14.2.6	Configure Status of Reminder	78
14.2.7	Notification Server Log	79
14.2.8	Offline Mode	80
14.2.9	Data Receiving Mode	81
14.2.10	Summary of GrowControl Website	82
15	Summary	83
15.1	Perspective	83
15.2	Function Extension	83
15.3	Conclusion	84
16	Link to our Implementation	85

1 Member List & Workload

No.	Full Name	Student ID	Workload	Contribution
1	Pham Nguyen Minh Hieu	2252215	- Task planning - Front-end and back-end implementation - Hardware process implementation - AI prediction implementation - Report writing	100%
2	Nguyen Ngoc Khoi	2252378	- AI detection implementation - Report writing	100%
3	Tran Xuan Bach	2252381	- Front-end development	90%
4	Nguyen Doan Hai Bang	2252078	- AI detection implementation	80%
5	Le Quang Dang	2252152	- Front-end and back-end development	80%

Table 1: Member list and corresponding workload distribution

2 Introduction

2.1 Smart farm

Agriculture is a fundamental pillar of Vietnam's economy, providing livelihoods for millions of rural workers. However, the sector faces growing challenges due to climate change, including rising sea levels, floods, droughts, saltwater intrusion, and unpredictable weather patterns. These environmental threats severely impact traditional farming methods, which still rely heavily on manual labor, leading to inefficiencies, increased costs, and reduced crop yields.

To address these pressing issues, we introduce Yolo:Farm, a small-scale smart farming project that integrates automation, the Internet of Things (IoT), and artificial intelligence (AI) into modern agriculture. By leveraging advanced technologies, Yolo:Farm aims to enhance productivity, optimize resource management, and reduce reliance on manual labor, making farming more efficient and climate-resilient. This project serves as a practical demonstration of how Agriculture 4.0 can revolutionize traditional farming, paving the way for a more sustainable and technology-driven agricultural future.

2.2 AIoT

- Artificial Intelligence of Things (AIoT) is the combination of artificial intelligence (AI) technologies with the Internet of things (IoT) infrastructure, enabling devices not only to collect vast amounts of data but also to process and learn from it in real time.
- When AIoT is applied to the smart farm, real-time data acquisition and advanced AI analytics transforms the farm into a modern intelligent ecosystem.
- With this integration, early detection of crop diseases, forecasting weather impacts or any proactive decision-making and resource management is achievable, ultimately enhance yield, efficiency, and reduced waste.

3 User Requirements

3.1 Functional Requirements

3.1.1 Application aspects

- Users are allowed to access and manually monitor environment around the plant by controlling light, motor for irrigation.
- Users can get all the parameters such as temperature, humidity rate, light and moisture rate on the dashboard.
- Users can set, modify reminders, and threshold value.
- Users can get statistics of all activities applied on the plant.
- Users can set schedules to control the irrigation/motor modules.

3.1.2 Internet of Things (IoT) aspects

- IoT system automatically irrigates the plant if temperature, humidity, and light rate are outside of the thresholds.
- IoT system automatically handles device malfunctioning or out-of-bounds when such events occur (rule chain).
- IoT system allows user to control devices manually (RPC).
- IoT system can do the above in schedule.

3.1.3 Artificial Intelligence (AI) aspects

- Detect fire on the tree.
- Detect leaf condition.
- Predict the incoming value of sensor.
- Diagnose events that are about to exceed thresholds and make timely adjustments.
- Save results.

3.2 Non-functional Requirements

- System should respond to user actions within 10 seconds.
- Sensor data should update corresponding to the user defined time, 1 minutes by default.
- System up time should be at least 99.9%.
- Data must be encrypted (AES key, TLS 1.2+).
- Web application should have a simple, intuitive UI.
- Dash board should have a simple, intuitive UI, easy to control, extend.
- System recovery should be possible within 5 minutes in case of failure.

4 System Requirements

4.1 Use-case Description

Use case ID	UC-01
Use case name	Irrigate Plant
Actor	User, IoT system
Description	Users can control the irrigation system manually or automatically. They can adjust irrigation frequency, turn the system on/off, and set motor duration. The IoT system can also auto-adjust water volume based on temperature and humidity.
Trigger	TRIG-1: User input via dashboard. TRIG-2: System detects abnormal soil moisture.
Preconditions	PRE-1: IoT system is online. PRE-2: Sensors are connected. PRE-3: User has control permission.
Postconditions	POST-1: Irrigation completed as triggered. POST-2: Action logged for monitoring. POST-3: User is notified.
Normal Flows	1. User logs in and opens Control Panel. 2. System displays current irrigation mode. 3. User selects Schedule Mode and configures duration/on-off. 4. System processes and executes command.
Alternative Flows	<i>At Step 3:</i> ALT-1: User selects Normal Mode (On/Off). ALT-1.1: System irrigates based on sensor data. ALT-1.2: System starts irrigation.
Exceptions	<i>At Step 3:</i> E-1: If system fails to respond, an error is shown and logged.

Table 2: Use-case description: Irrigate Plant

Use case ID	UC-02
Use case name	Control light
Actor	User, IoT system
Description	The user can control the lighting system in the farm. The system allows users to turn lights on or off , and schedule operations. The IoT system also adjusts lighting based on environmental conditions such as light sensor levels.
Trigger	TRIG-1: The user initiates the control action from the dashboard. TRIG-2: The system detects low light conditions and adjusts automatically.
Preconditions	PRE-1: The IoT system is online and functional. PRE-2: The light sensor is connected to the IoT network. PRE-3: The user has the necessary permissions to control the lighting.
Postconditions	POST-1: The lighting settings are updated according to user input or environmental factors. POST-2: The system records the new light status for monitoring and analytics. POST-3: The user receives a confirmation message about the action taken.
Normal flows	1. The user logs into the system and navigates to the “Control Panel” section. 2. The system displays the current lighting status, including schedule and on/off state. 3. UChooses one of the following actions: turn light on/off, set a manual lighting duration. 4. The IoT system processes the command and executes the action. 5. The system confirms the action and updates the dashboard in real-time.
Alternative flows	ALT-1: User select “Schedule mode”: ALT-1.1: Defines when lights should turn on/off based on ambient light levels or by your setup schedule. ALT-1.2: Sets specific times for lights to turn on/off (e.g., lights on at 6:00 PM, off at 6:00 AM).
Exceptions	E-1: If the lighting system fails to respond, the system displays an error message and logs the issue. <i>At ALT-1.2:</i> E-2: If the schedule is set incorrectly (e.g., ON time is later than OFF time), the system prompts the user to correct it.

Table 3: Use-case description of Control light

Use case ID	UC-03
Use case name	Remind
Actor	User
Description	This use case allows users to set periodic reminders for greenhouse-related tasks. Users can customize reminders for different tasks, such as watering, fertilizing, equipment maintenance, or other custom to-dos. The system will store and trigger reminders accordingly, keeping the user aware of scheduled tasks and system status updates.
Trigger	TRIG-1: The user manually sets a reminder. TRIG-2: The system automatically triggers a pre-set periodic reminder.
Preconditions	PRE-1: The application is running normally. PRE-2: The device's time system is functioning correctly. PRE-3: The server is operational and capable of storing and retrieving reminders.
Postconditions	POST-1: A new reminder is successfully stored in the system. POST-2: The reminder notification is triggered at the scheduled time. POST-3: The user is informed of the reminder and may take action accordingly.
Normal flows	1. The user selects "Reminder" from the main menu. 2. The system displays a list of current reminders. 3. The user selects "Add New Reminder" from the options menu. 4. The system presents a reminder creation form. 5. The user fills in the form and presses "Confirm". 6. The system validates the input and saves the new reminder. 7. The reminder is added to the list and will trigger notifications as scheduled.
Alternative flows	None
Exceptions	E-1: If the user presses "Confirm" without entering a title and/or scheduled time, a warning message is displayed and requires the user to fill again.

Table 4: Use-case description of Remind

Use case ID	UC-04
Use case name	Display Sensor Indices
Actor	User, IoT system, Website
Description	The system collects data from sensors and displays them on the dashboard in real time.
Trigger	TRIG-1: User selects the Dashboard from the website.
Preconditions	PRE-1: Devices are connected to the IoT system. PRE-2: System is online and fully operational.
Postconditions	POST-1: Dashboard shows updated sensor readings.
Normal Flows	1. User logs in and accesses the dashboard. 2. System retrieves and displays sensor data.
Alternative Flows	None
Exceptions	None

Table 5: Use-case description: Display Sensor Indices

Use case ID	UC-05
Use case name	View Statistics
Actor	User, IoT system
Description	Users can view aggregated statistics based on recorded environmental data collected over time.
Trigger	TRIG-1: User selects the Dashboard to access statistical data.
Preconditions	PRE-1: System has recorded sufficient telemetry data.
Postconditions	POST-1: System shows module activity frequency. POST-2: Displays average values of environmental parameters.
Normal Flows	1. User logs in and selects “Dashboard”. 2. System processes and displays statistics.
Alternative Flows	None
Exceptions	<i>At Step 2:</i> E-1: If no data is available, system shows error and logs it.

Table 6: Use-case description: View Statistics

Use case ID	UC-06
Use case name	View server log/notifications
Actor	User, IoT system
Description	The IoT system notifies user when user login or turn on/off motor, setting threshold value
Trigger	TRIG-1: Log in and Log out. TRIG-2: Control motor. TRIG-3: Setting threshold values.
Preconditions	PRE-1: The system is connected to the IoT system.
Postconditions	POST-1: Notifications appear.
Normal flows	<i>Those actions would be for the user to view notifications.</i> 1. User chooses "View server log/notifications" button. 2. All notifications is shown.
Alternative flows	None
Exceptions	None

Table 7: Use-case description of View notifications

Use case ID	UC-07
Use case name	Detect illness
Actor	User, IoT system
Description	AI system can analyse plant illness through photos taken by the camera.
Trigger	TRIG-1: AI system receives a photo from camera.
Preconditions	PRE-1: The system is connected to the AI system. PRE-2: The system is connected to the IoT system.
Postconditions	POST-1: The AI system uses machine learning to detect plant illness.
Normal flows	1. The AI system analyses the photo taken by the camera. 2. Result is saved to the database.
Alternative flows	None
Exceptions	None

Table 8: Use-case description of Detect objects

Use case ID	UC-08
Use case name	Detect fire
Actor	User, IoT system
Description	AI system can analyse fire through photos taken by the camera.
Trigger	TRIG-1: AI system receives a photo from camera.
Preconditions	PRE-1: The system is connected to the AI system. PRE-2: The system is connected to the IoT system.
Postconditions	POST-1: The AI system uses machine learning to detect fire in environment.
Normal flows	1. The AI system analyses the photo taken by the camera. 2. Result is saved to the database.
Alternative flows	None
Exceptions	None

Table 9: Use-case description of Detect objects

Use case ID	UC-09
Use case name	Detect next sensor value
Actor	User, IoT system
Description	Use AI models to predict possible sensor value to be appear based on current and past value of sensor.
Trigger	TRIG-1: AI system receives a previous data set.
Preconditions	PRE-1: The system is connected to the AI system. PRE-2: The system is connected to the IoT system.
Postconditions	POST-1: The AI system uses machine learning to predict sensor value.
Normal flows	None
Alternative flows	None
Exceptions	None

Table 10: Use-case description of Sensor value prediction

4.2 Use-case Diagram

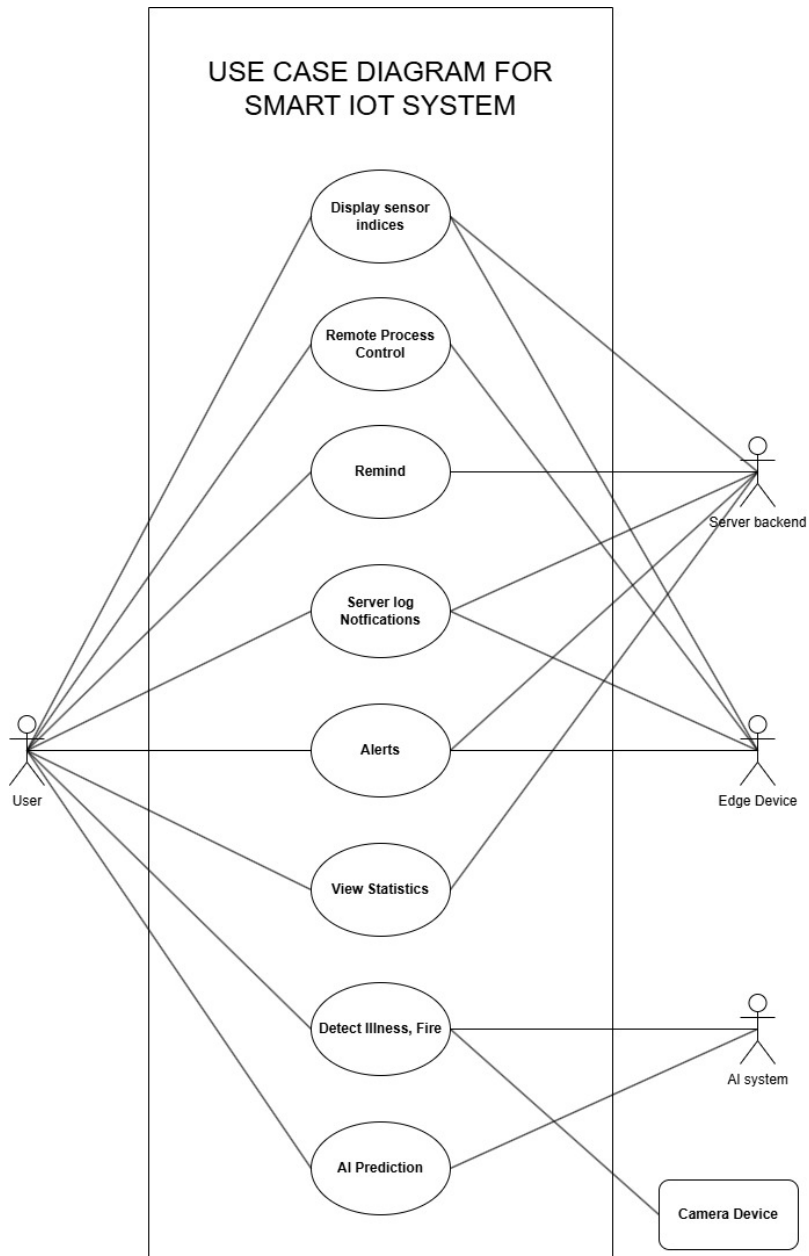


Figure 1: Use case Diagram

5 System Device

In this section, we describe the hardware at the “edge” of our AIoT smart-farm system and the gateway that bridges those devices to coreIOT thingboard

5.1 Microcontroller & Edge Devices

Each node in the smart-farm is built around an ESP32-based microcontroller running an RTOS. This real-time environment guarantees that sensor readings (temperature, humidity, soil moisture, light intensity) and actuator commands (LED RGB strips, water pumps, fan motor) are sampled and executed with precise timing and minimal jitter.

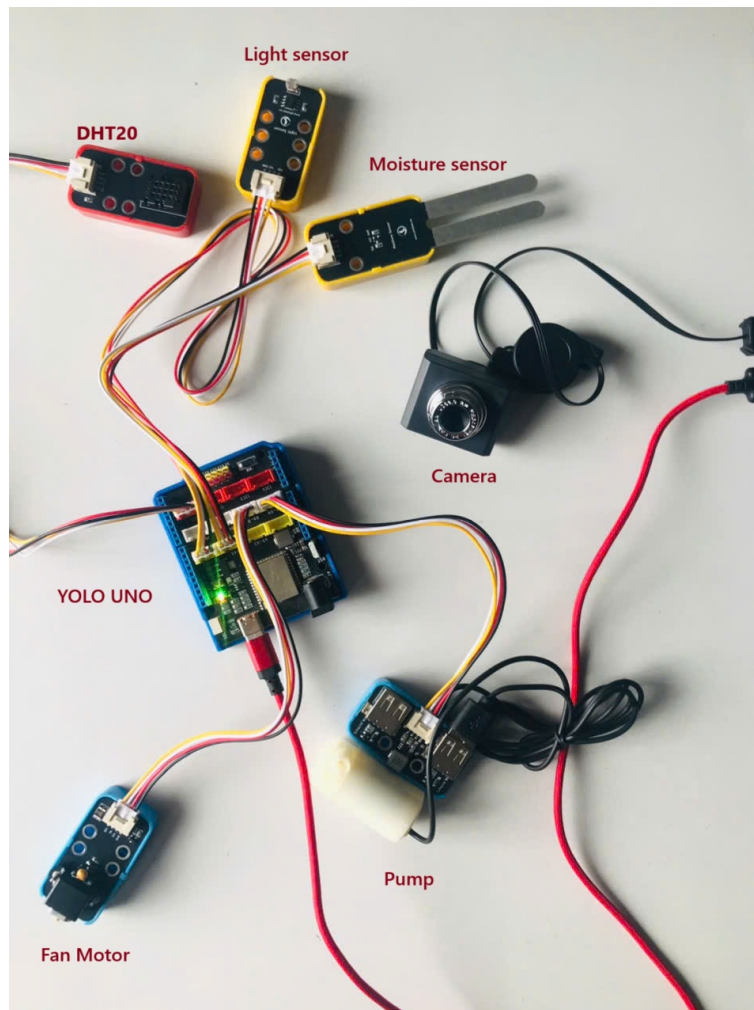


Figure 2: Device Description

5.2 Device Information

Yolo Uno Board: ESP32-based mini programmable board with built-in WiFi/Bluetooth.

- Program via online Scratch or MicroPython; Platformio for advanced use. Connect peripherals.
- Acts as the central controller for IoT tasks.

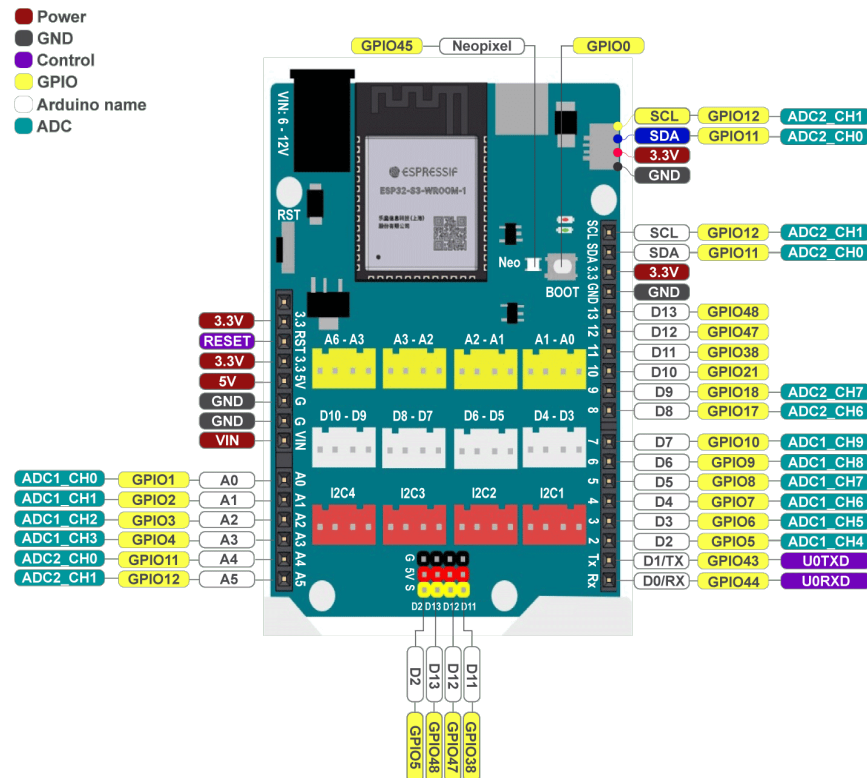


Figure 3: Device Description

DHT20 Sensor: Temperature and humidity sensor module.

- Connect to Yolo Uno via proper pins and use sensor libraries.
- Monitors environment temperature and humidity.

LED Device: Built-in LEDs on Yolo Uno.

- Program using built-in display functions.
- Mainly used for testing connection.

Light Sensor: Sensor for measuring ambient light intensity.

- Connect to input pin and read values via sensor commands.
- Assesses lighting conditions for monitoring crop growth.

Moisture Sensor: Sensor for measuring the humidity of dirt.

- Connect to input pin and read values via sensor commands.
- Assesses moisture conditions for monitoring crop growth.

Water Pump: Submersible pump for irrigation.

- Interface via the relay module and activate with programmed commands.
- Automates water delivery for smart irrigation.

Fan Motor: Motor for turn on when temperature too high.

- Interface via the relay module and activate with programmed commands.
- Automates turn on the fan delivery for smart irrigation.

Camera: Camera module for AI detection

- Integrate with Yolo Bit or gateway using appropriate libraries for image capture.
- Captures real-time images for surveillance and AI analysis.

Laptop (Gateway/Edge device): Computer for coding and interfacing with thingboards/-coreIOT services.

- Connect Yolo Uno via USB/WiFi; run the development and monitoring environment.
- Serves as the development platform and data gateway.

5.2.1 Input

This project utilizes Yolo Uno (ESP32) to collect data from input devices/sensors through a pin and transfer it to the IoT gateway. For our smartfarm system, the input sensors and wiring consist of:

- GPIO 48: Led on board
- I2C4: DHT20
- A1-A2: Soil moisture sensor
- A0-A1: Light sensor

5.2.2 Output

The IoT gateway will transfer the information to the output devices via Yolo Uno by pin:

- D7-D8: Fan motor.
- D9-D10: Two pumps, one in used.

6 Gate Way/Edge Device

Edge device: An edge device is a piece of hardware that processes data near the source of data generation (like sensors, cameras, or other IoT devices) rather than sending all data to a centralized server or cloud for processing. - *Key Features:*

- Located at the “edge” of a network, close to the data source.
- Performs local data processing, analysis, or decision-making.
- Reduces latency, bandwidth usage, and dependency on cloud services.

- *Why Use Edge Devices?*

- Faster response time (e.g. in real-time systems)
- Improved privacy (less data sent to cloud)
- Lower network load
- Continued operation even when offline

Gateway: The gateway sits between the RTOS-based microcontrollers and our coreIOT thing-board, in charge of three primary responsibilities.

6.1 Command & Control Bridging

The gateway subscribes to per-device MQTT topics via the CoreIoT/ThingsBoard broker, listening for RPC callbacks or user-issued control messages via our custom dashboard. When a user turns on/off a pump or adjusts a light, the gateway immediately translates those MQTT commands into local action calls, either issuing an internal event or signaling the microcontroller via serial port to trigger the required output.

6.2 Telemetry Aggregation & Streaming

All raw sensor measurements and local inferences (e.g. growth stage predictions) are streamed in real-time to coreIOT, which publishes this data over MQTT - ensuring that the cloud always has up-to-date time-series telemetry. As it receives the raw sensor results, it also predicts the next possible values based on deep learning and sends these values to coreIOT along with the current values.

6.3 Local Alarm and Multimode Control

To handle situations that need to be resolved without network latency, the gateway also runs an alarm manager. If a remote measurement exceeds a safety limit or a sensor reports an error code, the gateway will generate a local alarm and notify the cloud, and take timely action by turning on or off cooling devices, motors, etc.

6.4 Gateway Operations

The gateway mediates between our RTOS-based edge controllers and the cloud over MQTT, providing low-latency command/control, real-time telemetry streaming, and autonomous scheduling. To coordinate these concurrent responsibilities without blocking, we model each background loop as an independent finite-state machine (FSM).

6.4.1 Background Task Finite-State Machines

Each FSM encapsulates one of the gateway's three core loops: telemetry listening, schedule dispatching, and RPC handling. Figures 4–5 illustrate their state transitions and error recovery paths.

6.4.1.1 Telemetry Listener FSM

- **Listening → Processing:** When raw bytes are received, parse them into sensor readings and detect any device error codes.
- **Processing → Validation:** If the analysis is successful, we will proceed to process the values to see if they are feasible or not. If they are feasible, we will use those values to predict the next possible values.
- **Validation → Publish:** When the data is valid, publish over MQTT. When publishing fails, switch to Reconnect.
- **Reconnect → Listening:** Attempt to reestablish the cloud connection, notify when there are errors, then return to Listen.

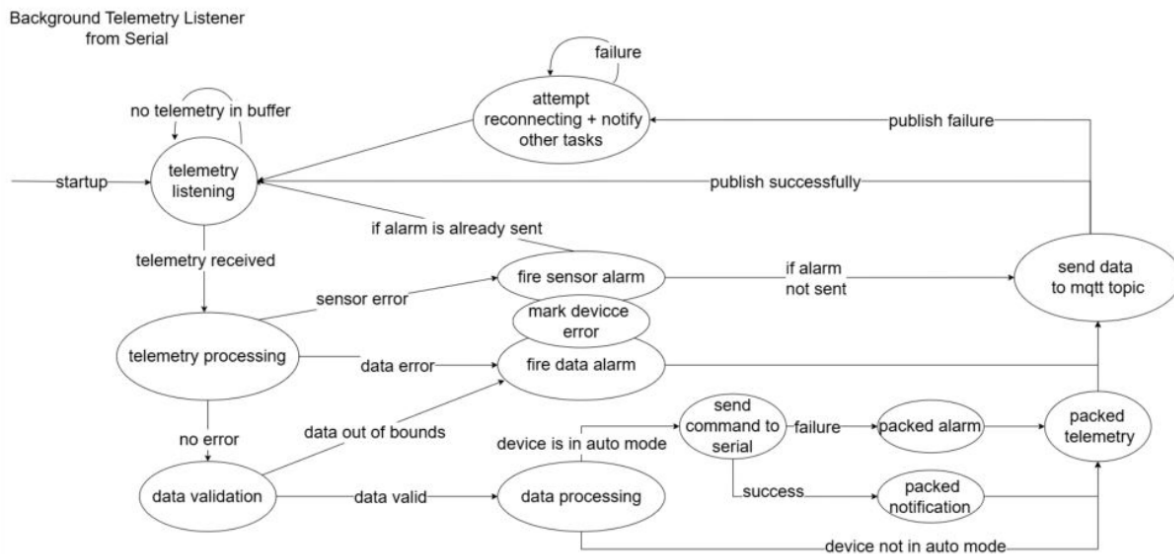


Figure 4: Telemetry Listener FSM

6.4.1.2 Telemetry Processing in coreIOT

- **Received:** When receiving the values uploaded from the edge device, the system will first check the value according to the conditions of the rule chain, if it is too satisfactory, it will be updated.
- **Processing:** If the values exceed the threshold value, a warning will be issued and return to **[Received]** process.

6.4.1.3 Telemetry Processing in Web App

- **Received:** The website will get the value from coreIOT to display through rest API.
- **Processing:** If the values exceed the threshold value, a warning will be issued and return to **[Received]** process.

6.4.1.4 RPC Listener FSM (Finite State Machine)

- **Startup:** Subscribes to the device RPC topic, transitions to **ListenRPC**.
- **ListenRPC → ProcessRPC:** On receiving a valid RPC JSON.
- **ProcessRPC → SerialCommand:** Maps method to serial command or prepares error.
- **SerialCommand → RPCResponse:** Sends ACK-based response or error.
- **RPCResponse → ListenRPC:** Waits for the next RPC.

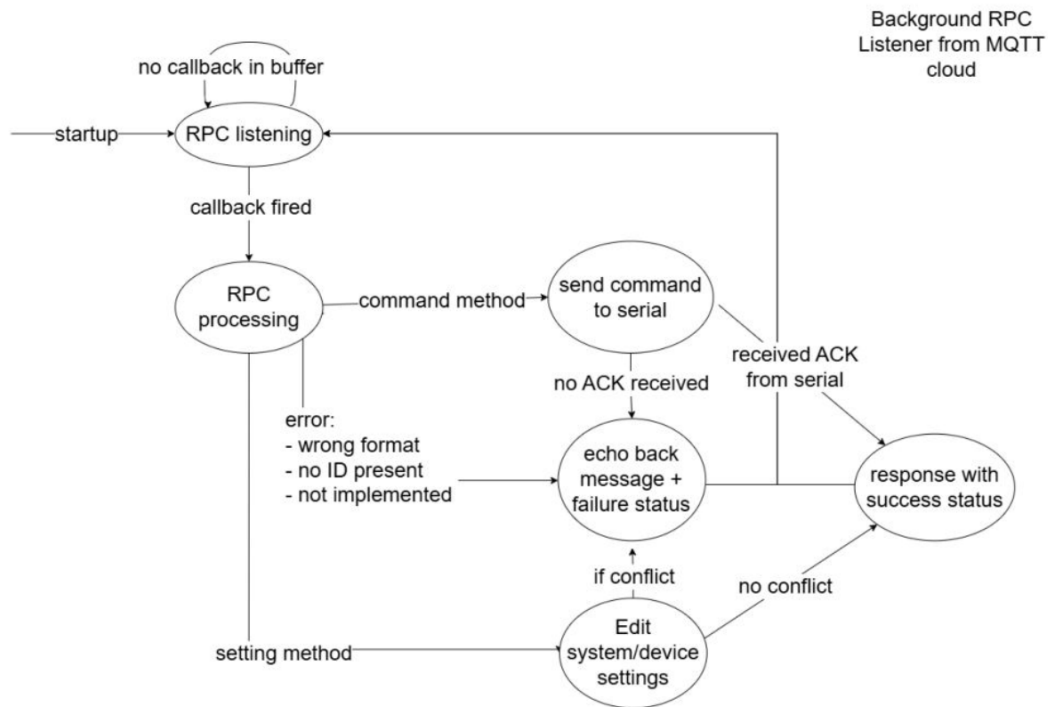


Figure 5: RPC Listener FSM

7 MQTT server

It is important to recognize that an MQTT server, or a broker, is the heart of any IoT messaging architecture. In this smart-farm AIoT system, CoreIoT, a Cloud IoT platform built on top of ThingsBoard, serves as the cloud-hosted MQTT broker, managing device connectivity, message routing, persistence, and telemetry visualization.

7.1 MQTT Broker

Every MQTT broker is responsible for three core functions:

- Client Connectivity & Session Management
- Topic-based Message Routing
- Quality of Service (QoS) Guarantees

In an agricultural IoT context, devices (gateways, sensors, controllers) connect over unreliable networks. MQTT's lightweight protocol and three QoS levels (0: at most once, 1: at least once, 2: exactly once) ensure that critical measurements—soil moisture, temperature, pump control signals are delivered reliably without overwhelming constrained devices.

7.2 ThingsBoard

ThingsBoard excels at end-to-end IoT lifecycle management combining MQTT, REST APIs, rule chains, and dashboards—several other technologies in the MQTT ecosystem (like EMQX, HiveMQ, and Mosquitto). Together, these components form the backbone of a reliable, low-latency, and secure pipeline for device telemetry, command/control, and system monitoring.

In this exercise, our main goal is to develop a custom dashboard and alert system from end to end, so we will take advantage of all the powerful features of ThingsBoard's built-in UI and rule-engine functionality. At the same time, we will use ThingsBoard purely as a lightweight bridge between our backend and our web app. There will be 2 UIs in this exercise, 1 in coreIoT, 1 in the Web App.

- RPC callbacks for issuing control commands back to devices
- Telemetry ingestion and persistence (MQTT-based streaming into its time-series store)
- A handful of auxiliary services.

7.3 CoreIoT

CoreIoT is an open-source professional IoT platform designed for education, research, and prototype projects, offering a free-to-deploy solution for academic institutions and developers.

CoreIoT uses ThingsBoard's Community Edition as its MQTT broker and event-driven core, featuring underlying components:

- **MQTT/HTTP Gateway:** Devices communicate over MQTT or HTTP(S), with CoreIoT handling message routing and storage.
- **Device profile:** A Device Profile in ThingsBoard Core IoT is essentially a template or blueprint that defines a set of common behaviors, settings, and configurations for one or more similar devices. By creating a device profile, you can avoid repeating the same configuration on each individual device, enforce consistent data schemas, and streamline onboarding (provisioning) of new devices.
- **Rule Engine:** Incoming telemetry triggers customizable rule chains for data enrichment, transformation, or forwarding.
- **Alarm:** An Alarm represents a time-stamped, stateful notification that indicates a certain condition (e.g., a threshold breach, device offline, or custom business logic) has occurred on one of your IoT entities (devices, assets, or customers). Alarms are central to building monitoring and alerting workflows in ThingsBoard.
- **Dashboard:** A dashboard is a visual interface that allows you to monitor, analyze, and control your devices and assets in real-time. It provides charts, gauges, maps, tables, and widgets that are connected to telemetry data or device attributes.
- **Entity aliasing:** An Entity Alias is simply a logical name (or “shortcut”) that you give to one or more ThingsBoard entities (Devices, Assets, Customers, Dashboards, etc.) so that your widgets, alarms, rule chains, and scripts can refer to them by alias instead of hard-coding UUIDs. By using aliases, you make your dashboards and rule chains far more reusable and maintainable—if the actual device or asset behind an alias ever changes, you only need to update the alias configuration in one place instead of hunting down every widget or script that referred to a specific UUID.
- **SDK & Client Libraries:** CoreIoT’s client SDK supports Arduino and Espressif platforms, abstracting MQTT/HTTP details and enabling quick sensor and actuator integration.

8 System Design

8.1 Behavior Diagram

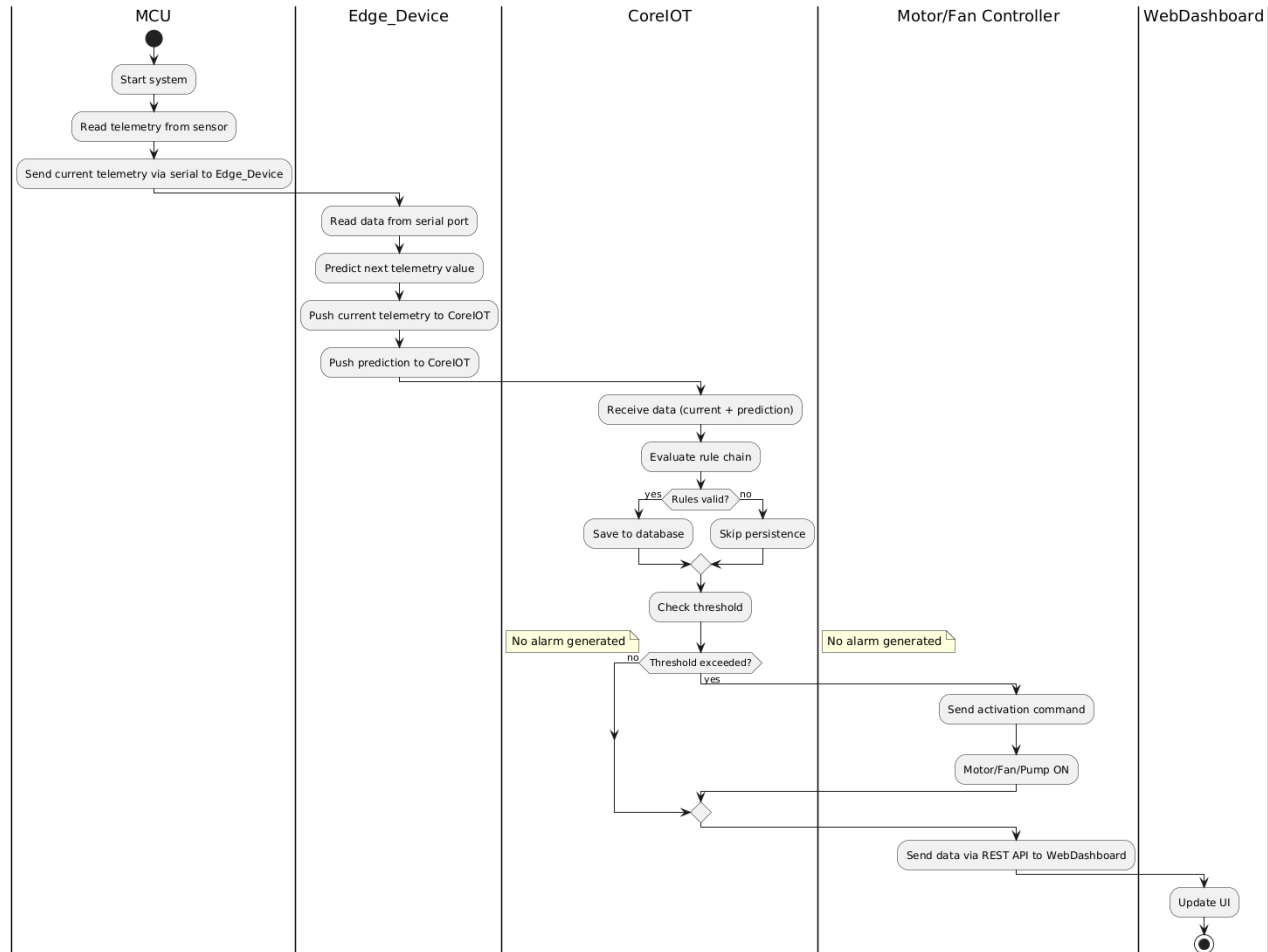


Figure 6: Behavior Diagram of Telemetry workflow

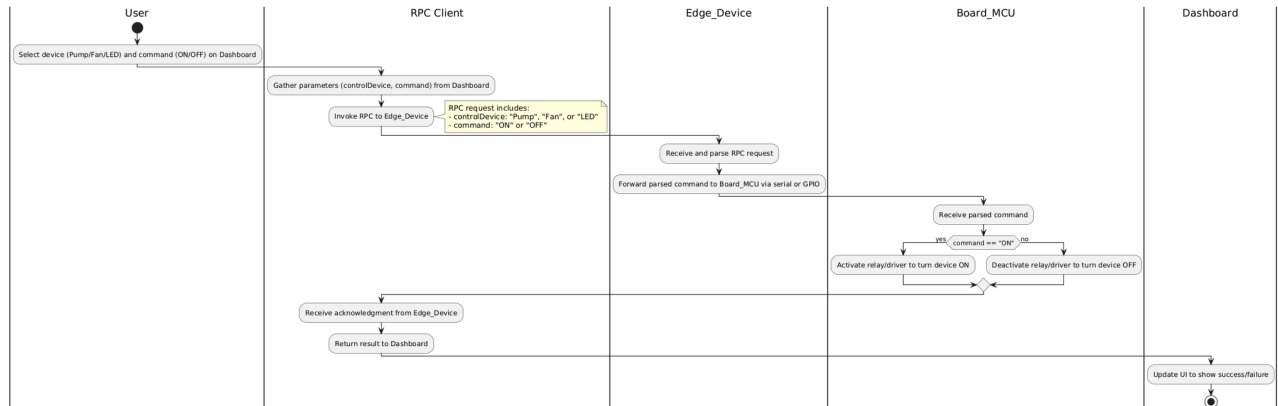


Figure 7: Behavior Diagram of RPC workflow

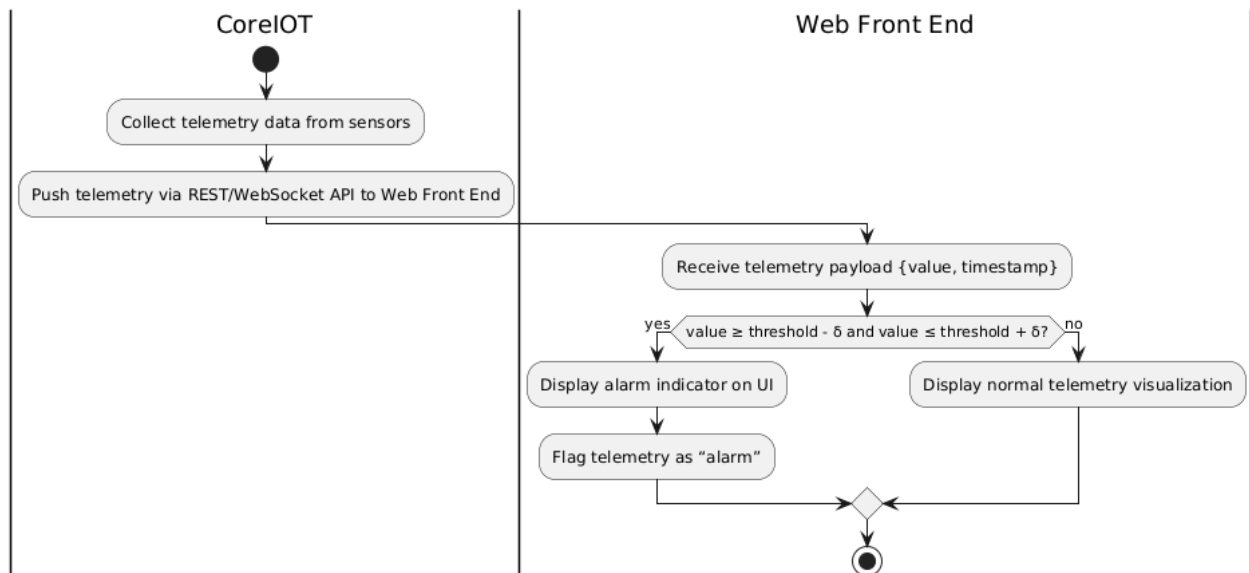


Figure 8: Behavior Diagram of Alarm workflow

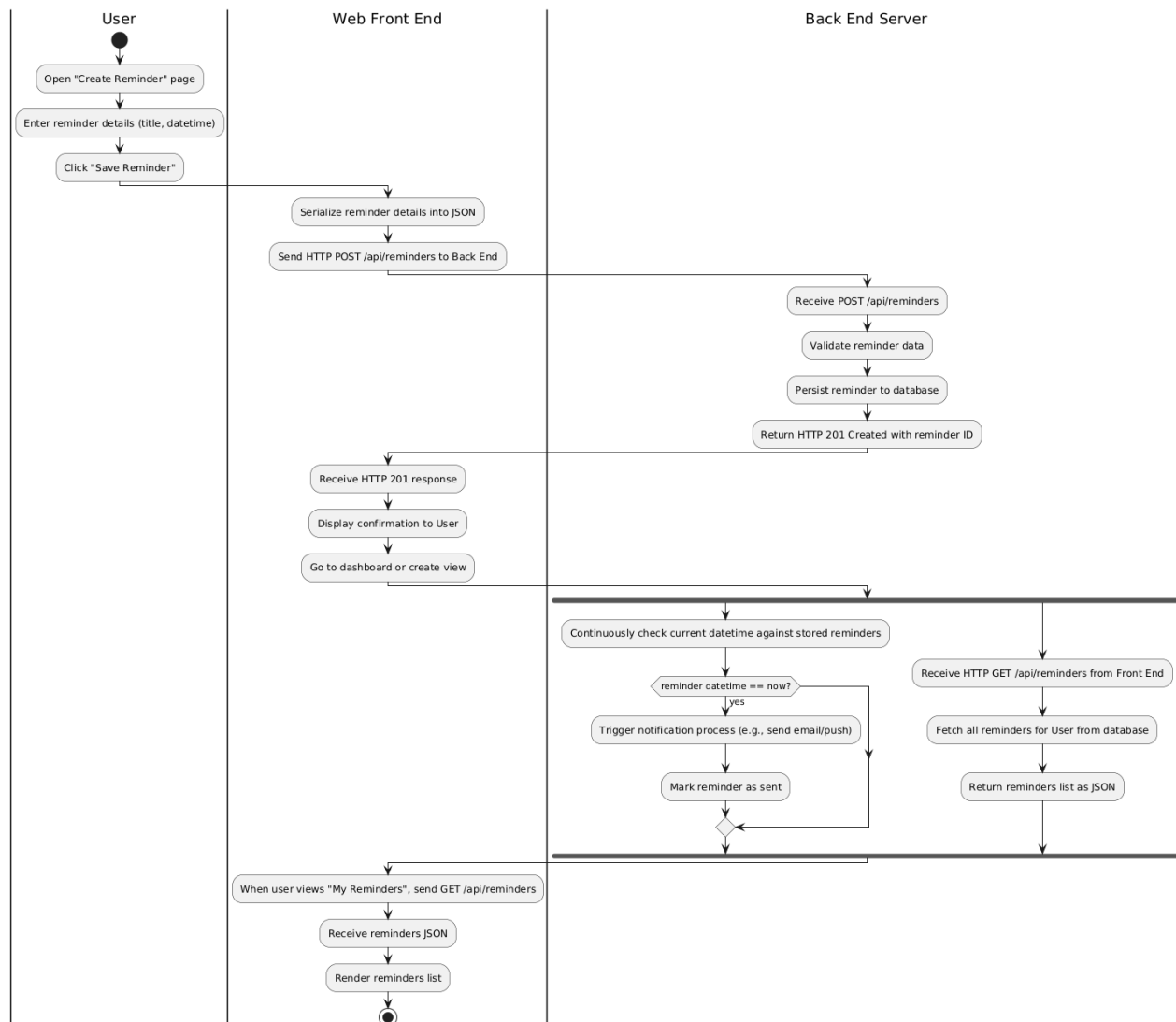


Figure 9: Behavior Diagram of Remind workflow

8.2 Sequence Diagram

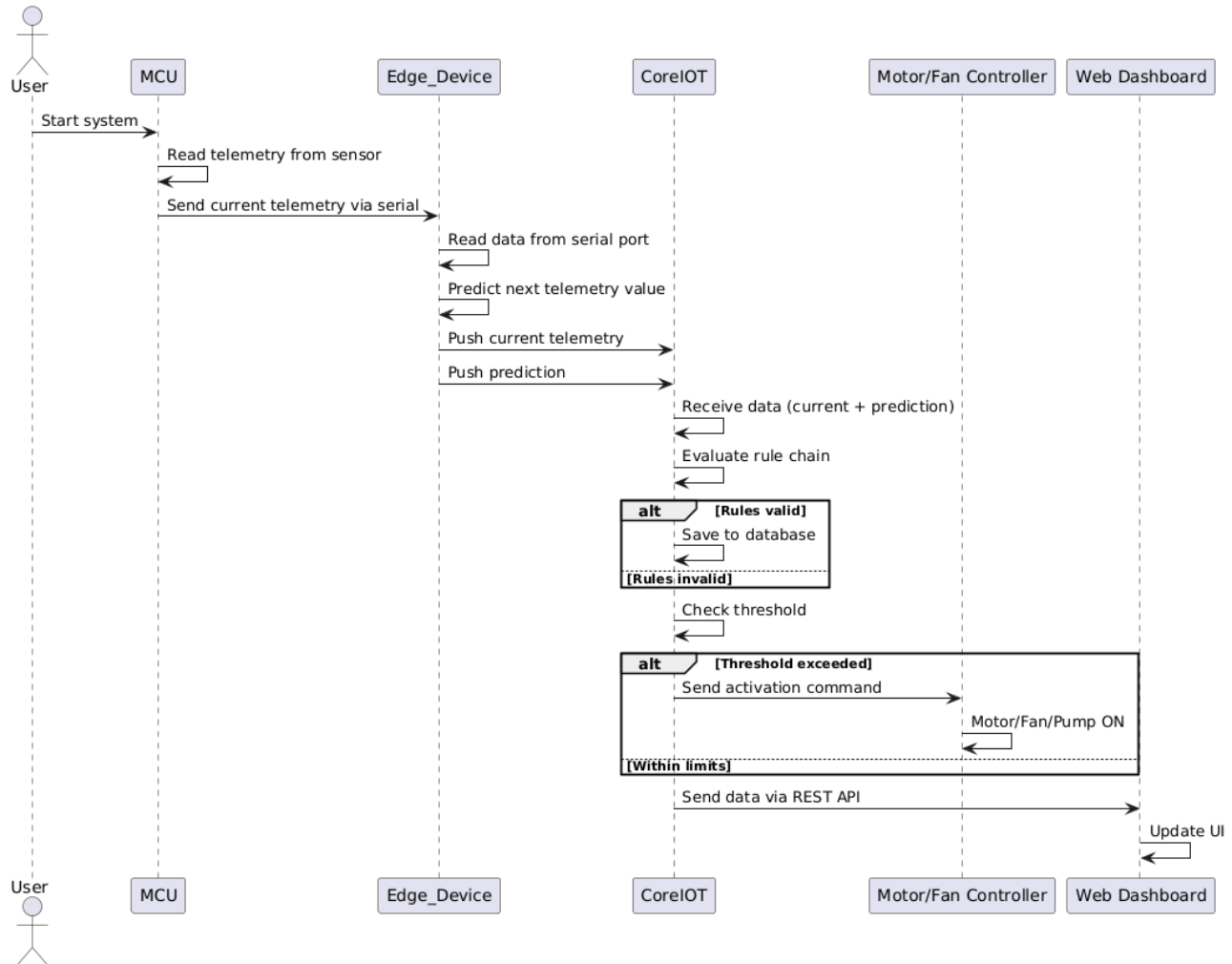


Figure 10: Sequence Diagram of Telemetry workflow

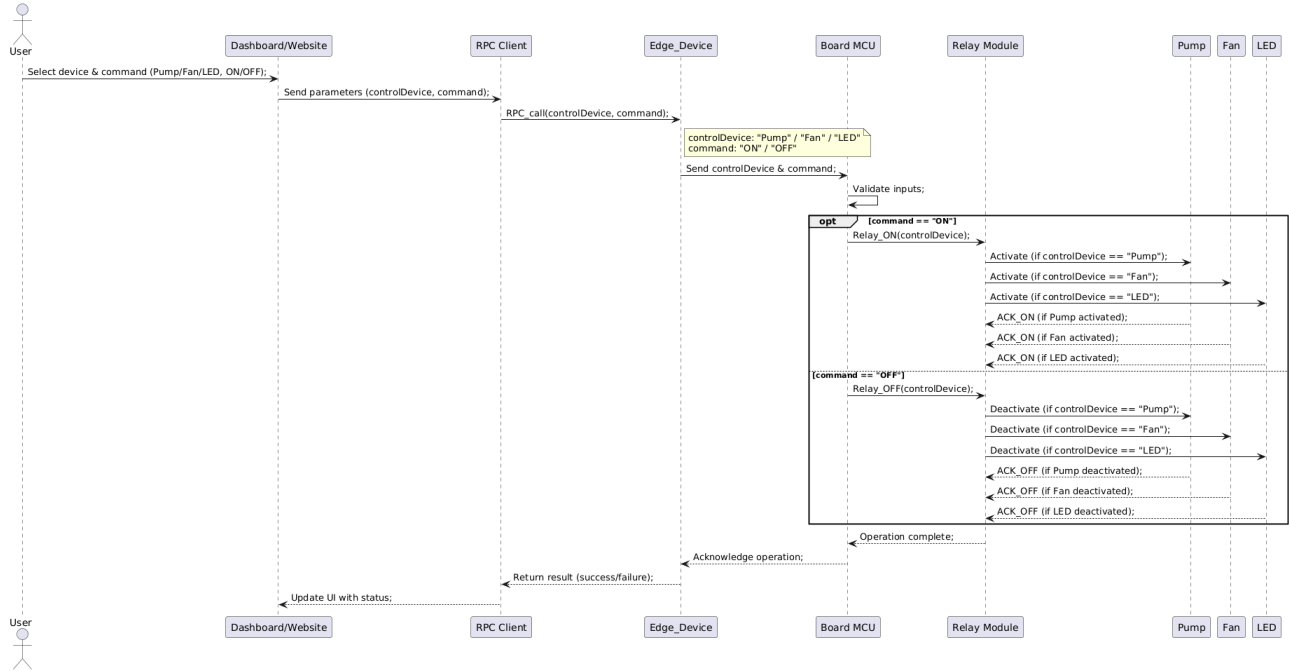


Figure 11: Sequence Diagram of RPC workflow

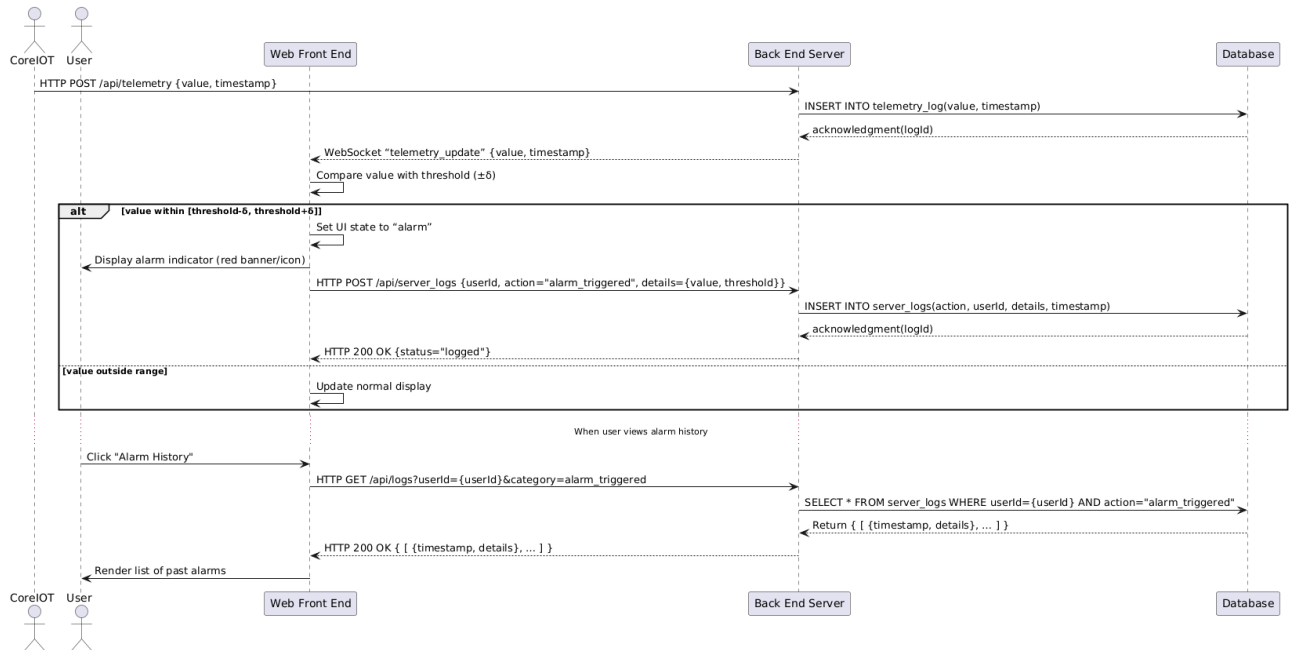


Figure 12: Sequence Diagram of Alarm workflow

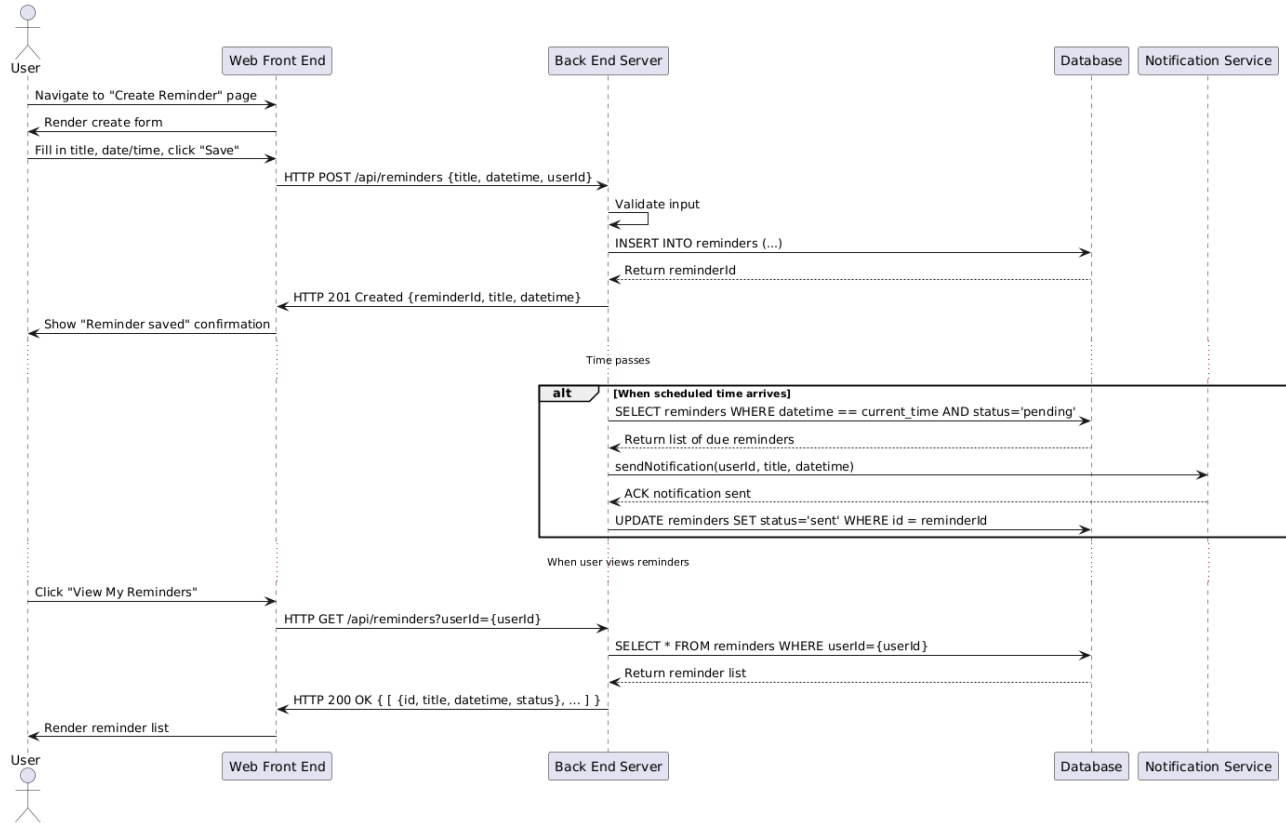


Figure 13: Sequence Diagram of Remind workflow

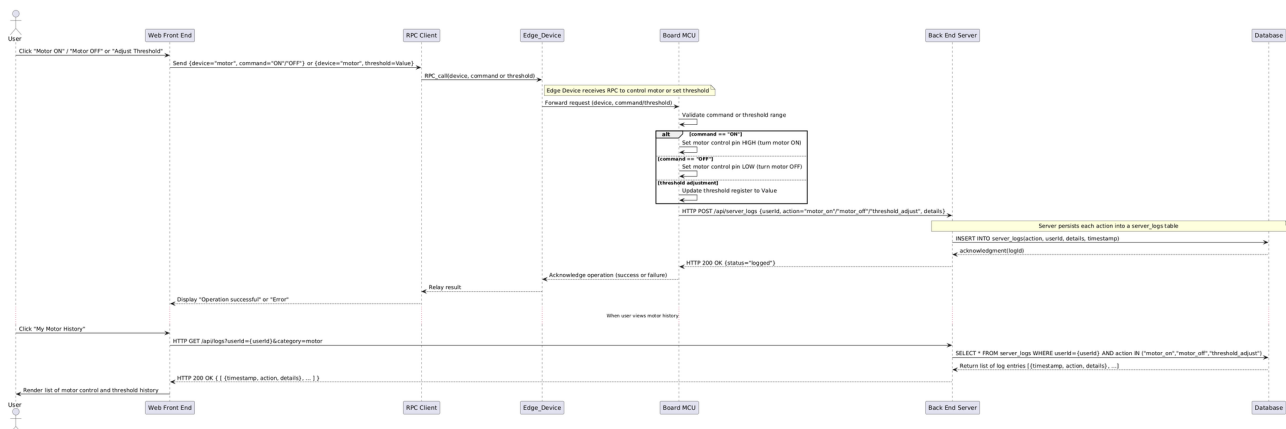


Figure 14: Sequence Diagram of Server log (action in control) workflow

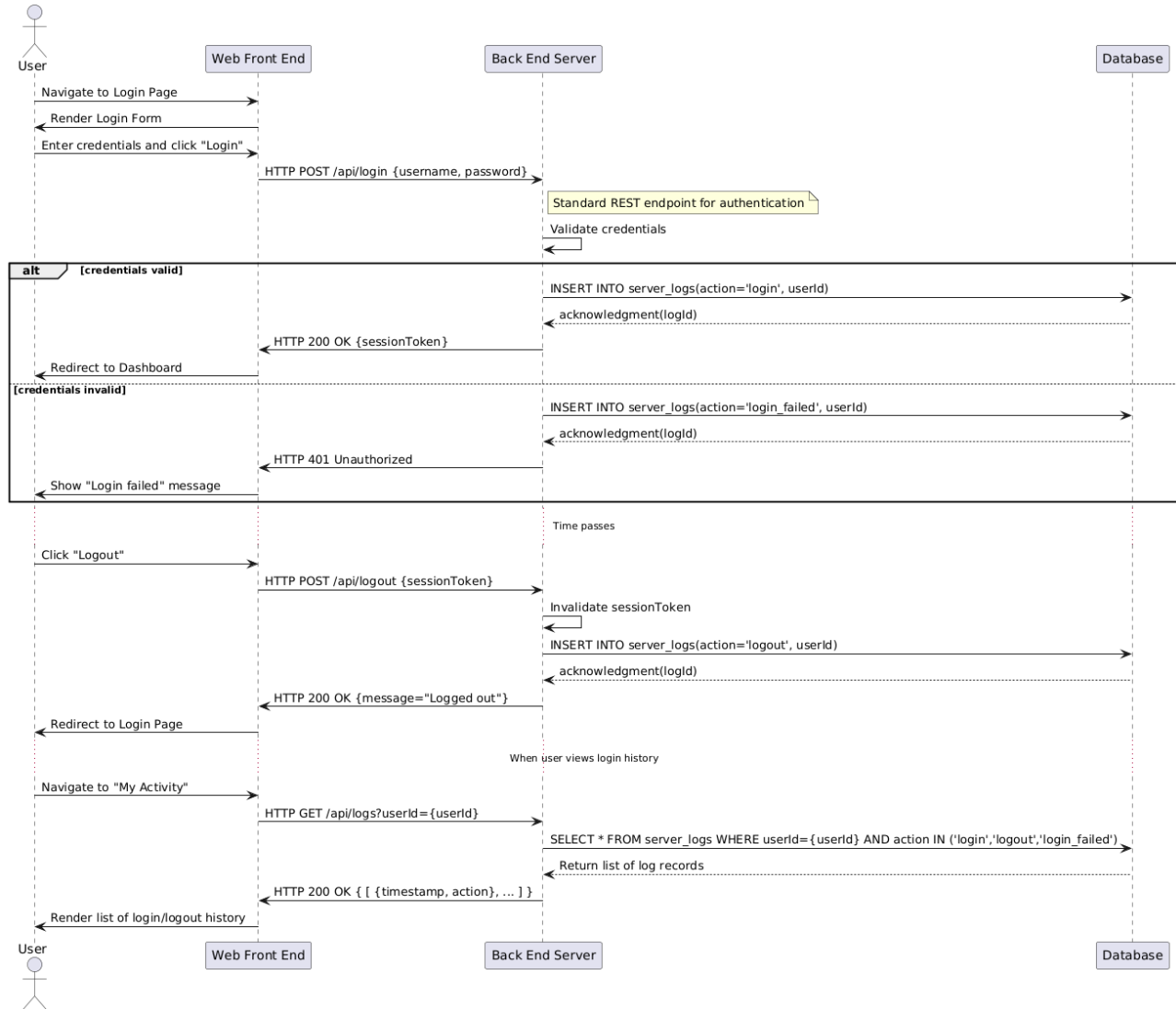


Figure 15: Sequence Diagram of Server log (login/logout) workflow

8.3 Class Diagram

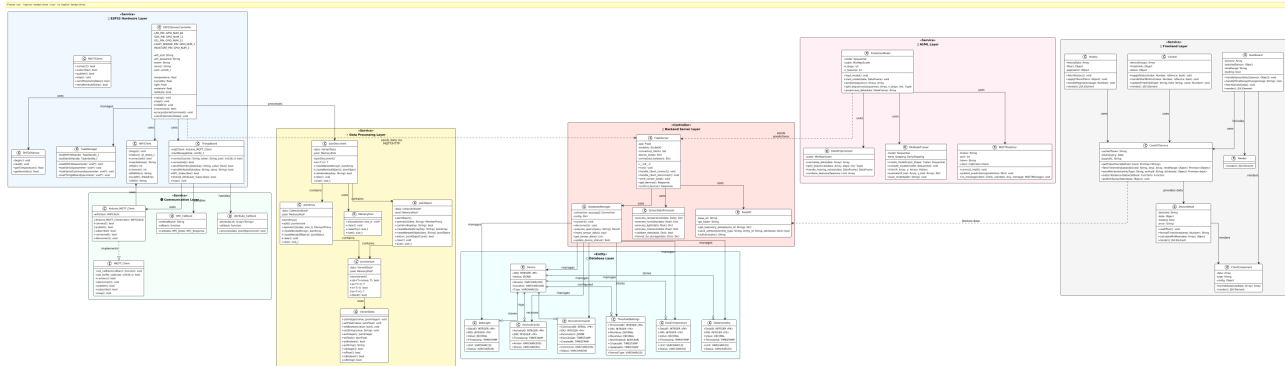


Figure 16: Class Diagram of Smart farm system

Below is a detailed walkthrough of each of the major “blocks” (or layers) in the diagram, followed by a high-level description of how they all fit together into one cohesive system. For clarity, I have roughly divided the diagram into:

- ESP32 Hardware and Communication Layer
- Data Processing Layer
- Database Layer
- Backend Server Layer
- ML/AI Layer
- Frontend Layer

Each of these “modules” corresponds to one colored region in the class diagram. After explaining each layer in isolation, I’ll describe the overall data flow—how a new sensor reading ultimately shows up in the front end (and how the ML model can be invoked along the way).

More Details: Layered System Architecture

- **ESP32 Hardware Layer:** This layer corresponds to the firmware executing on the ESP32 (or equivalent microcontroller). Its primary responsibility is to interface with physical sensors (e.g., fire detectors, leaf-color sensors), manage network connectivity via Wi-Fi and/or MQTT, and transmit raw measurement data to the subsequent layer.
 - **ESP32Controller:** Serves as the central coordinator on the microcontroller. It initializes Wi-Fi connectivity, configures sensor GPIO pins, and schedules periodic tasks. Internally, it invokes lower-level client classes such as `HTTPClient`, `WiFiClient`, or `MQTTClient` to package sensor readings into JSON (or binary) payloads and dispatch them to the cloud server.

- **HTTPClient / WiFiClient / MQTTClient:** These classes wrap Arduino’s networking libraries, exposing simplified methods such as `connect()`, `publish(topic, payload)`, and `get()`. They are invoked by `ESP32Controller` to push telemetry data or receive server-issued commands.
- **TaskManager:** Implements an RTOS-based scheduler on the ESP32. It manages recurring tasks (e.g., “acquire temperature reading every five seconds,” or “scan for fire/leaf detectors every minute”) by allowing registration of callback functions (e.g., sensor-read and data-send callbacks). This design decouples time-critical sensing logic from the main `loop()` function, ensuring deterministic execution.
- **Communication Layer:** After raw sensor values are obtained by the hardware layer, they must be reliably transmitted to backend services. Depending on configuration, the system can employ:
 - **Arduino_MQTT_Client:** A wrapper over an MQTT library (e.g., `PubSubClient`) to manage broker connections. It handles subscription/publishing semantics, QoS levels, and callback registration for remote-procedure calls or attribute updates originating from the server.
 - **HTTPClient (Hardware Layer reuse):** If MQTT is not adopted, the ESP32 may issue HTTPS/HTTP POST requests containing JSON payloads to a RESTful endpoint. Methods such as `get()` and `post()` are used to transmit data and process response codes. Both MQTT and HTTP implementations implement a common “service interface,” enabling protocol selection via a single configuration change (e.g., modifying a flag in `config.h`).
- **Data Processing Layer:** Operating on an edge device or within a lightweight microservice (e.g., Python/Node-based), this layer ingests raw byte streams or JSON messages and transforms them into structured domain objects. Its responsibilities include:
 - **DataCollector:** Listens on a TCP socket, REST endpoint, or MQTT subscription to receive incoming payloads. It parses raw JSON (or binary) into strongly typed entities (e.g., `SensorReading` objects).
 - **DataAnalyzer (DataProcessor):** Upon reception of a new `SensorReading`, it applies domain-specific logic—such as threshold checks, unit conversions, initial outlier detection—and optionally enriches data with metadata (e.g., timestamp, device location). Processed readings may be enqueued for storage or forwarded to downstream layers.
 - **ViewModel / DTO:** Constructs a lightweight data transfer object containing only the fields required by persistence layers or API serialization (e.g., temperature, humidity, `deviceId`, `timestamp`). This minimizes payload size and decouples presentation concerns from raw data structures.

- **Backend Server Layer:** Serving as the core of the cloud or on-premises infrastructure, this layer exposes REST and/or MQTT APIs, orchestrates database operations, and invokes AI/ML routines when necessary.
 - **MainServer (e.g., AppServer or Node.js):** Bootstraps all essential services, including HTTP listeners and MQTT brokers (if applicable). It maintains references to components such as **DatabaseManager** and **ServiceDataProcessor**, routing inbound requests to the appropriate handlers.
- **HttpRequestHandler / MqttMessageHandler:** These specialized classes decode incoming requests—e.g., “POST /sensor” or an MQTT message—deserialize JSON into **SensorReading** objects, and forward them to **ServiceDataProcessor** for further business logic execution.
- **DatabaseManager:** Encapsulates a database connection pool (e.g., JDBC, ORM framework, or direct SQL client). It exposes CRUD methods—**saveReading()**, **getAllDevices()**, **updateDeviceStatus()**, etc.—and ensures that raw SQL queries reside solely within this class, thereby enforcing a clear separation of concerns.
- **Backend Model Classes (POJOs/Entities):** Represent primary domain entities (e.g., **Device**, **Reading**, **ThresholdRule**). These classes are used by **ServiceDataProcessor** and **DatabaseManager** to shuttle data between memory and persistence. They contain minimal business logic—for example, a method like **Reading.isFireDetected()** that compares reading values against stored thresholds.
- **Database Layer:** Defines the physical schema for persistent storage:
 - **Device:** Stores metadata (**deviceId**, **modelName**, **deployment location**, **operational status**). Establishes a one-to-many relationship with sensor readings and connection records.
 - **SensorReading (ReadingEntity):** Each table row captures a single sensor measurement (e.g., **temperature**, **humidity**, or **leaf color**). Fields include **id** (PK), **deviceId** (FK), **timestamp**, **value**, and an optional **type** column.
 - **ThresholdRule / AlertConfig:** Persists user-defined thresholds (e.g., “fire if temperature $\geq 60^{\circ}\text{C}$ ” or “alert if leaf discoloration $\geq 50\%$ ”). **ServiceDataProcessor** consults these rules to determine when an alert should be emitted.
 - **DeviceConnection:** Logs active sessions or tokens, enabling the system to track which devices are currently online. Typical fields include **lastSeen**, **connectionId**, and **isOnline**.

Each of these classes is a simple POJO or ORM-annotated entity containing only getters, setters, and minimal constructors—no complex business logic. Their sole purpose is to mirror the relational schema.
- **AI/ML Layer:** Responsible for advanced analytics, including computer vision and time-series forecasting:

- **ImageProcessor:** When the system supports image capture (e.g., for leaf-color analysis), this component reads image files (JPEG/PNG) or raw byte streams from a camera. It performs preprocessing steps such as resizing, normalization, and denoising prior to inference.
 - **ModelRunner (YoloInferenceEngine):** Loads a pre-trained YOLO (or similar) model (e.g., using TensorFlow, PyTorch, or ONNX) into memory. It provides a method such as `runInference(imageTensor)`, which returns raw bounding boxes and confidence scores.
 - **ResultAnalyzer:** Interprets the raw output from the model (e.g., bounding boxes, class probabilities) and translates them into higher-level events (e.g., “fire detected at sector X” or “leaf-color intensity detected as ‘yellow’”). It may also compute summary statistics (e.g., percentage of pixels classified as ‘leaf’, average leaf-color intensity).
 - **MQTTPublisher / AlertNotifier:** When the AI model identifies a significant event (fire or unhealthy leaf), this component publishes an alert onto an MQTT topic or invokes a REST callback on the main server. This design enforces a clean separation of concerns: the AI/ML layer focuses solely on inference and alert generation, without requiring knowledge of downstream processing.
- **Frontend Layer:** The web or mobile client resides here, delivering dashboards, charts, and control panels by consuming APIs exposed by the Backend:
 - **Dashboard / ControlPanel:** Implemented with frameworks such as React or Angular. These components fetch data from endpoints (e.g., `/api/readings/latest`) or subscribe to restAPI streams. They render real-time visualizations (e.g., line charts for temperature over time) and status indicators (e.g., “Device Online/Offline,” “Fire: Yes/No”).
 - **History / DataContainer:** These services request historical readings (e.g., data from the past 24 hours) and transform them into a format suitable for charting libraries (e.g., Chart.js or D3). Functionality includes pagination, filtering (e.g., “Display readings from Device A between 08:00 and 10:00”), and data export (e.g., CSV).
 - **ChartComponent:** A reusable widget that accepts a property such as `dataPoints = [{x: timestamp, y: value}, ...]` and renders line, bar, or scatter charts. It may also listen for new data over MQTT to update charts in real time.

Data Flow Overview

• Sensor Acquisition on ESP32

- At fixed intervals, the `ESP32Controller` schedules a callback via the `TaskManager` to read data from temperature, humidity, soil moisture, and light sensors.
- If a camera module is connected, the `ESP32Controller` captures an image, converts it to JPEG, and encodes it in Base64.
- The collected measurements are assembled into a JSON-formatted payload containing the device identifier, timestamp, temperature, humidity, light intensity, soil moisture, and, if present, the Base64-encoded image.

• Transmission via Communication Layer

- The `ESP32Controller` invokes either the MQTT client or the HTTP client, depending on compile-time configuration.
- If MQTT is selected, the client publishes the JSON payload to a designated telemetry topic with quality of service set to at least once, retrying on failure.
- If HTTP is selected, the client issues a POST request containing the JSON payload to the server's REST endpoint, verifying that a successful (2xx) response is returned or logging the error for retry.

• Reception and Preliminary Processing

- On the server side, an *MqttMessageHandler* subscribed to the telemetry topic or an *HttpRequestHandler* listening on the sensor API endpoint receives the incoming JSON payload.
- The handler deserializes the JSON into a domain object (e.g., a `SensorReading` instance) containing fields for device ID, timestamp, numerical sensor values, and optional image data.
- Initial validation includes:
 - * Verifying that the device identifier exists in the Device registry.
 - * Ensuring that numerical values fall within expected physical bounds (for example, temperature between -40°C and 85°C).
 - * If image data is present, confirming that it is valid Base64 and nonempty.
- If validation fails, the handler returns a 400 Bad Request (for HTTP) or publishes a failure notification (for MQTT), and logs the error via the logging subsystem.

• Business Logic in `ServiceDataProcessor`

- `ServiceDataProcessor` receives the validated `SensorReading`.
- It checks numerical values against threshold rules and enqueues alerts if breached.

- If image data is present, it saves the image to a temporary file and invokes `AIInferenceEngine`.
- It converts `SensorReading` into a persistence entity and calls `DatabaseManager.saveReading`.
- If any alert is triggered, it publishes to the “alerts/deviceId” MQTT topic.

- **AI/ML Analysis (if image present)**

- The `AIInferenceEngine` consumes the image file path from its work queue and executes a model inference (for example, using a YOLO-based detector) on the image.
- The `ResultAnalyzer` interprets raw detections—such as “fire detected with confidence 0.92” or “leaf discoloration at 45 percent”—and produces an `AIResult` object containing the device identifier, timestamp, result type, confidence score, and any relevant metadata.
- The `AIResult` is returned to `ServiceDataProcessor.processAIResult`, which persists it via `DatabaseManager.saveAIResult`. If the result indicates a critical event (for example, fire detected), an immediate alert is published to the corresponding MQTT topic notification is sent to the frontend.

- **Database Persistence**

- `DatabaseManager.saveReading` inserts a new row into the `SensorReading` table, including fields for device identifier, timestamp, temperature, humidity, light, moisture, and an optional reference to the stored image path.
- `DatabaseManager.saveAIResult` inserts a new row into the `AIResult` table, recording the device identifier, timestamp, result type, confidence score, and associated metadata.
- Both methods return success or failure; on transient errors (such as deadlocks), they automatically retry.

- **Frontend Delivery**

- `ServiceDataProcessor` sends real-time messages (JSON with `deviceId`, timestamp, sensor values, and AI flags).
- Frontend components (`Dashboard`, `ControlPanel`) subscribe to that channel and update charts and status indicators upon receiving new data.
- For historical data, users call `/api/readings?deviceId=<id>&start=<startTimestamp>&end=<endTimestamp>`. The backend returns a JSON array, which the `History` component converts into data points for the `ChartComponent`.

9 Technology

9.1 Front-end

9.1.1 HTML

HTML is the standard markup language used to create and structure content on the web. It is used to define the layout and structure of a webpage, using a variety of elements such as headings, paragraphs, links, images, and more.

HTML offers several key features:

- **Tag-based structure:** HTML uses tags to define the structure of a webpage, organizing elements such as headings, paragraphs, lists, and links for browsers to easily interpret.
- **Cross-device compatibility:** HTML ensures that webpages are accessible across different devices and operating systems.
- **Interactive elements:** HTML supports interactive components like forms and multimedia, making web pages more dynamic and engaging.
- **Semantic elements:** Tags such as `<header>`, `<footer>`, and `<article>` provide meaning to the structure of a page, improving SEO and accessibility.
- **Integration with other technologies:** HTML works well with CSS for styling and JavaScript for interactivity, enabling developers to create rich web experiences.

Advantages of HTML

- **Universal compatibility:** HTML works across all web browsers and is easy to learn, making it an essential skill for web developers.
- **Organization and structure:** It helps structure content like text, images, and multimedia, working in conjunction with CSS to create visually appealing designs.
- **Helpful tools:** Many tools are available to make HTML coding easier and more efficient.
- **Improved SEO:** A well-structured HTML document enhances website visibility in search engines.

Disadvantages of HTML

- **Static content:** HTML alone creates static pages. To add dynamic content, additional technologies like JavaScript are required.
- **Complexity in management:** Large and complex HTML structures can be challenging to manage.
- **Error-prone:** Small mistakes in HTML code can lead to significant issues with webpage rendering or functionality.

9.1.2 CSS

CSS is a stylesheet language used to describe the presentation of a document written in HTML or XML. It controls the layout, colors, fonts, and overall appearance of the webpage. It allows web developers to apply styles to web pages for a more attractive and user-friendly experience.

Advantages of CSS

- **Responsive Design:** CSS enables responsive design through media queries, allowing websites to adjust seamlessly to various screen sizes.
- **Animations and Transitions:** With CSS, developers can create smooth and interactive animations, enhancing the visual experience of a website.
- **Reusability:** CSS stylesheets can be reused across multiple pages or projects, promoting efficiency and ease of maintenance.

Disadvantages of CSS

- **Learning Curve:** Mastering advanced concepts such as the box model, positioning, and animations can be challenging for beginners.
- **Performance Impact:** Large CSS files with complex selectors and deep nesting can negatively impact page load speed.
- **Debugging Challenges:** The cascade effect and conflicting styles can make debugging CSS difficult and time-consuming.

9.1.3 SCSS

SCSS is a syntax of SASS (Syntactically Awesome Stylesheets), an extension of CSS. SCSS provides advanced features such as variables, nested rules, and mixins, which help developers write more maintainable and reusable styles. It is a superset of CSS, meaning all valid CSS code is also valid SCSS.

Advantages of SCSS over plain CSS:

- **Maintainability and Readability:** By using variables, mixins, and nesting, SCSS keeps the code following the principle of DRY (Don't Repeat Yourself) and easy to navigate. When a color or spacing variable changes, we update it in one place rather than hunt down every instance in hundreds of CSS rules.
- **Modularity:** Partials + imports encourage splitting large stylesheets into logical chunks (e.g., `_variables.scss`, `_typography.scss`, `_buttons.scss`). In contrast, a single, monolithic CSS file can become unwieldy as the project grows.
- **Powerful Abstractions:** Mixins, functions, and control directives let you abstract complex patterns (e.g., fluid typography, responsive grid helpers) into reusable constructs, saving time and reducing errors.

- **Compiler-Level Error Checking:** Since SCSS must be compiled, many syntax mistakes (missing semicolons, unbalanced braces, undefined variables) are caught at build time instead of causing silent styling bugs in the browser.
- **Ecosystem and Community:** SCSS has a mature ecosystem of frameworks (Bootstrap, Foundation, Bulma) and community packages that provide prebuilt mixins (e.g., Bourbon, Compass) and “utility” functions for complex tasks (like automatic vendor prefixing with Autoprefixer).
- **Backward Compatibility:** All valid CSS is valid SCSS. You can gradually adopt SCSS in an existing CSS project by simply renaming `.css` files to `.scss` and replacing certain rules with variables or mixins as needed.

Disadvantages and Trade-offs:

- **Build Step Required:** Browsers cannot natively interpret SCSS. You need a build tool (Webpack, Gulp, Rollup, Vite, or a standalone `sass` CLI) to compile SCSS → CSS. This adds configuration overhead, Node.js dependencies, and another failure point in your CI/CD pipeline.
- **Potential Over-Abstraction:** It is easy to create excessively complex mixins or deeply nested rules that harm performance or readability. Over-nesting generates large, deeply specific CSS selectors that can lead to specificity wars and bloated files.
- **Debugging Compiled CSS:** Once SCSS is compiled to CSS, line numbers and file references change. Although source maps help, you sometimes need to spend a moment tracing back from the generated CSS to the original SCSS, especially during complex nesting or when multiple imports are involved.
- **Versioning & Dependencies:** Different SCSS libraries and mixin collections (e.g., Bourbon vs. Compass vs. Susy) can conflict. You must manage your package versions carefully to avoid breaking changes when upgrading SASS or third-party SCSS frameworks.

9.1.4 JSX

JSX (JavaScript XML) is a syntax extension for JavaScript, most commonly used with React to describe user interfaces in a declarative, component-based style. Although it looks like HTML, JSX is ultimately transformed—by tools like Babel—into standard `React.createElement(...)` calls before it runs in the browser. Embedding UI markup directly within JavaScript code helps keep component logic and presentation in one place, making it easier to reason about how data flows through the application.

Comparison to “plain” JavaScript/HTML:

- **Declarative Syntax vs. Imperative DOM Manipulation:** JSX allows you to write code such as

```
<button onClick={handleClick}>  
  Click me  
</button>
```

instead of manually creating DOM nodes and attaching event listeners in pure JavaScript (e.g., `document.createElement('button') + button.addEventListener('click', ...)`). This declarative style often leads to more readable code, because the structure of the UI is apparent at a glance.

- **Component Co-location of Markup + Logic:** By embedding JSX inside a component's JavaScript file (for example, `Header.jsx`), you colocate the rendering logic (data/state handling) with the markup that depends on it:

```
function Header({ user }) {  
  return (  
    <header className="site-header">  
      <h1>Welcome, {user.name}</h1>  
    </header>  
  );  
}
```

In contrast, with “plain” JavaScript, you often have separate HTML templates or string concatenation to assemble the DOM, and you must manually keep them in sync with your application's state.

- **Compile-Time Error Checking:** Unlike writing raw HTML in strings, JSX—when processed by a transpiler (e.g., Babel)—catches syntax errors (such as mismatched tags, missing closing brackets, or invalid JavaScript expressions inside `{ ... }`). This early feedback prevents many run-time mistakes that might otherwise slip through if you built your DOM imperatively.
- *Tighter Integration with Modern Tooling:* Because JSX must be compiled, you typically set up a build pipeline (with Babel and Webpack, or Vite) that also enables linting, minification, and fast hot-reload. This contrasts with plain JavaScript/HTML pages, which can be served without a build step—at the cost of foregoing JSX's expressiveness and the ecosystem of React-specific lint rules, ESLint plugins, and IDE autocompletion.
- **Advantages of JSX:**
 - **Enhanced Readability:** UI structure is described using HTML-like syntax next to JavaScript expressions (`{}`), making it easier to visualize component hierarchies.
 - **Reusability via Components:** You can define small, self-contained components (e.g., `<Button />`, `<Modal />`, etc.) and reuse them—with props—for a consistent look-and-feel.

- **Rich Tooling / IDE Support:** Most editors understand JSX and can offer syntax highlighting, autocompletion (e.g., recognizing valid HTML attributes, React hooks), and inline error markers before you even save/compile.
- **Easier Conditionals and Iteration:** Embedding JavaScript directly in markup allows for constructs like:

```
{items.length > 0
  ? items.map(item => <li key={item.id}>{item.text}</li>)
  : <li>No items found</li>}
```

rather than manually building DOM trees or concatenating HTML strings.

- Disadvantages of JSX:

- **Requires Build Tooling:** Browsers do not natively understand JSX, we must configure a transpiler (Babel, SWC, etc.) and bundler (Webpack, Vite) to convert it into valid JavaScript at build time.
- **Less Separation of Concerns:** Since JSX mixes markup and logic, it can feel counterintuitive to those who prefer strictly separating HTML, CSS, and JavaScript into distinct files. In very large components, the combination of lengthy render code and business logic can become unwieldy.
- **Toolchain Overhead:** For small projects or simple static sites, introducing JSX and React may be overkill—requiring Node.js, npm, and configuration when a plain HTML/CSS/vanilla-JS approach might suffice.

9.2 Back-end

9.2.1 Python

Python is a high-level, interpreted programming language known for its simplicity and readability. It is widely used for web development, data analysis, artificial intelligence, machine learning, and automation tasks. Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

9.2.2 REST API

REST API is a set of architectural principles for designing networked applications. It uses HTTP requests to perform CRUD operations (Create, Read, Update, Delete) on resources, typically in the form of JSON or XML data. REST APIs are stateless and scalable, commonly used in web services to allow communication between client and server.

Key Features of REST APIs

- **Stateless:** Each client request must contain all the information needed for the server to process it, as the server does not store session state.
- **Client-Server Architecture:** REST APIs follow a client-server model, allowing the client and server to operate independently, which enhances scalability.
- **Cacheable:** Server responses can be marked as cacheable or non-cacheable, improving performance by reducing redundant processing.
- **Uniform Interface:** REST APIs adhere to conventions like consistent URL paths, standardized HTTP methods, and status codes, ensuring clear and efficient communication.
- **Layered System:** REST APIs can be implemented in multiple layers, enhancing scalability and security.

9.2.3 JSON Backend

A JSON backend refers to a server-side system (often part of a web service or API) that sends and receives data in JSON (JavaScript Object Notation) format. JSON is a lightweight, text-based data interchange format that's easy for humans to read and write and easy for machines to parse and generate. In a JSON-based backend, the backend:

- Accepts HTTP requests (typically RESTful)
- Parses JSON payloads
- Processes the request using business logic
- Sends JSON responses back to the client (web, mobile, etc.)

Key Features of a JSON Backend

- **Data Format:** Uses JSON for input/output data format.
- **Stateless Communication:** Commonly follows RESTful principles.
- **Language Agnostic:** JSON is supported across all major programming languages.
- **Ease of Use:** Simple syntax, similar to JavaScript object notation.
- **Integration Friendly:** Works well with web frontends, mobile apps, and IoT devices.
- **Interoperability:** JSON is widely adopted and compatible with most modern APIs and tools.

Advantages of a JSON Backend

- **Lightweight and Efficient:** Smaller payloads compared to XML and fast to parse and generate.
- **Easy to Read and Write:** Human-readable syntax and simple to debug and log.

- Broad Support: Built-in support in most programming languages (Python, JavaScript, Java, Go, etc.)
- Easily handled by frontend frameworks like React, Vue, Angular.
- Cross-Platform Communication: Works seamlessly between client and server, regardless of tech stack.
- Ideal for REST APIs: Standard format for modern RESTful services.

Disadvantages of a JSON Backend

- No Data Types for Complex Structures: Doesn't support types like Date, Decimal, or Binary natively.
- Everything is text; you often have to encode/decode special types manually.
- No Schema Enforcement: Unlike XML with XSD, JSON doesn't have strict built-in schema validation (though tools like JSON Schema exist).
- Security Concerns: Susceptible to attacks like JSON injection or CSRF if not handled securely.
- Needs proper sanitization and validation of incoming data.
- Larger than Binary Formats: For data-heavy applications, binary formats like Protocol Buffers or MessagePack may be more efficient.
- Performance Bottleneck in Large Payloads: Parsing large JSON files can be memory-intensive compared to more compact formats.

10 Incorporate CNNs for Prediction

Predicting environmental parameters like humidity, temperature, lighting, and moisture is essential for enabling proactive, data-driven decisions in various domains, including smart homes, agriculture, industrial automation, and environmental monitoring. Hence, we implement an artificial intelligence (AI) prediction model for environmental sensor data, focusing on humidity, temperature, lighting, and moisture.

10.1 Dataset Description

The dataset is taken from SML2010, available at the [UCI Machine Learning Repository](#), collected from a monitor system in a domotic house over approximately 40 days, sampled every minute and smoothed with 15-minute means. This dataset is suitable for time-series prediction tasks and includes potential missing values, with time information in UTC and headers in Spanish. This dataset provided includes approximately 2700 rows in the training set and 1400 in the test set, with features such as moisture, CO2_room, Relative_humidity_(dinning-room), humidity, Lighting_(dinning-room), lighting, and meteorological data including Meteo_Rain, Meteo_Sun_dusk, and Meteo_Wind.

Table 11: Dataset Features and Descriptions

Feature	Description	Unit
Date, Time	Timestamp in UTC	–
moisture, humidity	Indoor relative humidity measurements	%
CO2_room	Carbon dioxide level in the room	ppm
Relative_humidity_(dinning-room)	Relative humidity in dining room	%
Lighting_(dinning-room), lighting	Lighting levels in dining room and in general	Lux
Meteo_Rain, Meteo_Sun_dusk, Meteo_Wind	Meteorological data (rain %, sun dusk, wind)	Various
Temperature	Indoor and outdoor temperature	°C
Day_of_the_week	Day of the week (1=Monday, 7=Sunday)	–

Id	Date	Time	moisture	CO2_room	Relative_hur	humidity	Lighting_(dir	lighting	Meteo_Rain	Meteo_Sun	Meteo_Winc	Meteo_Sun	Meteo_Sun	Meteo_Sun	Meteo_Sun	Outdoor_reli	Day_of_the	temperature
0	13/03/2012	11:45	216.56	221.92	39.9125	42.415	81.665	113.52	0	623.36	1.42625	9690.24	12604.2	95436.8	758.88	48.375	2	17.8275
1	13/03/2012	12:00	219.947	220.363	39.9267	42.2453	81.7413	113.605	0	623.211	1.592	11022	10787.2	95436.8	762.069	47.808	2	18.1207
2	13/03/2012	12:15	219.403	218.933	39.772	42.2267	81.424	113.6	0	622.656	1.89133	13960.5	9669.63	95398.6	766.251	47.432	2	18.4367
3	13/03/2012	12:30	218.613	217.045	39.776	42.0987	81.5013	113.344	0	622.571	1.828	18511.2	9648.13	95360.3	766.037	47.024	2	18.7513
4	13/03/2012	12:45	217.714	216.08	39.7757	42.0686	81.4657	113.034	0	622.4	2.36071	26349	9208.32	95354.9	762.743	45.4743	2	19.0414
5	13/03/2012	13:00	218.989	216.011	39.8107	42.0347	81.2427	113.6	0	622.4	2.06333	36511.7	8694.78	95354.9	752.683	45.5467	2	19.2773
6	13/03/2012	13:15	218.155	215.829	39.8613	41.9947	81.36	112.981	0	622.4	2.23067	49288.5	8501.25	95354.9	740.949	44.968	2	19.5367
7	13/03/2012	13:30	216.661	215.296	39.824	41.9947	81.024	111.259	0	621.803	1.924	62253.7	8687.62	95174.7	721.109	45.1813	2	19.7307
8	13/03/2012	13:45	217.461	214.549	39.9587	42.1253	81.104	83.3547	0	620.651	2.51067	71453.4	8888.32	91848.7	699.392	45.416	2	19.9693
9	13/03/2012	14:00	215.349	213.909	39.94	42.0427	81.1813	83.36	0	620.309	2.65333	80076.8	9046.02	87217.5	677.675	45.0213	2	20.1407
10	13/03/2012	14:15	213.888	213.312	39.9813	42.1787	61.0693	83.6107	0	619.755	2.82667	86682.3	9182.21	81720.7	648.427	44.36	2	20.3267
11	13/03/2012	14:30	215.413	213.739	40.3587	42.7067	51.2373	59.952	0	619.712	2.49467	88648.4	9347.07	75380.1	615.68	44.7947	2	20.4713
12	13/03/2012	14:45	216.021	222.528	40.9107	41.5147	43.3733	52.824	0	619.328	2.466	91406.3	9712.64	68515.2	582.869	43.984	2	20.9093
13	13/03/2012	15:00	216.256	219.68	41.1987	40.956	36.2653	45.6747	0	619.243	2.432	90472.4	9963.52	60604.4	544.533	44.5467	2	21.4253
14	13/03/2012	15:15	213.173	219.285	41.88	40.6347	34.476	40.16	0	619.157	1.732	88391.7	9655.3	52794.7	503.744	44.7813	2	21.8467
15	13/03/2012	15:30	211.093	217.589	43.2213	40.8413	31.1267	32.248	0	618.944	1.89133	89134.4	8651.78	44743.3	460.523	45.6773	2	22.1573
16	13/03/2012	15:45	211.157	215.136	43.3653	40.8867	21.442	23.44	0	618.645	1.64267	88462.7	7906.3	36479	415.744	45.5173	2	22.404
17	13/03/2012	16:00	209.419	214.24	42.744	40.3867	20.822	23.44	0	618.56	1.59067	85333.3	7433.22	28270.6	368.917	45.2373	2	22.7307
18	13/03/2012	16:15	209.408	213.632	42.216	39.9227	20.8233	23.44	0	618.56	1.732	84587.9	7175.17	21476.7	320.448	45.0747	2	22.9373
19	13/03/2012	16:30	209.547	215.029	41.976	39.6133	21.2633	22.244	0	618.517	1.59	78588.6	6831.1	16318.5	271.723	45.072	2	23.2
20	13/03/2012	16:45	212.715	217.067	41.52	39.084	20.4807	23.44	0	617.92	1.94733	74555.4	6515.71	12792.5	222.443	44.128	2	23.436

Figure 17: SML2010 Training Set

10.2 Basic Statistics and Machine Learning Concepts

The implementation involves time-series prediction, a statistical and machine learning task where past observations predict future values. Key statistical concepts include:

- **Descriptive Statistics:** Calculating mean, median, standard deviation, and range for features like humidity and temperature helps understand data distribution and variability, crucial for model input.
- **Correlation Analysis:** Examining correlations between features (e.g., humidity and temperature) can reveal dependencies, aiding feature selection or understanding interactions.
- **Time Series Analysis:** The data exhibits temporal dependencies, with trends, seasonality, and autocorrelation analyzed to capture patterns over time.

From a machine learning perspective, this is a supervised learning problem, specifically regression, as we predict continuous values (humidity, temperature, etc.). The chosen model, a Convolutional Neural Network (CNN), is typically used for image data but adapted here for 1D time-series data, leveraging its ability to capture local patterns. Key concepts include:

- **Neural Networks for Regression:** The CNN uses layers like Conv1D, MaxPooling1D, and Dense layers to learn from sequences, with ReLU activation and 'he_normal' initialization for better training.
- **Time-Series Forecasting:** The model predicts the next time step's values based on 3 previous steps, a common approach in environmental monitoring for proactive decision-making.
- **Data Preprocessing:** Normalization using MinMaxScaler ensures data is scaled between 0 and 1, and handling missing values (via forward fill) maintains data integrity.

These concepts ensure the model can learn from the dataset's temporal structure and predict future environmental conditions accurately.

We have implemented the AI prediction model with the following steps:

- **Data Loading and Preprocessing:**

- Loads training and test data from CSV files (`train.csv` and `test.csv`), checking for missing values and handling them via forward fill.
- Normalizes the data using `MinMaxScaler` to scale values between 0 and 1, essential for neural network training.
- Splits data into sequences of 3 time steps using a custom `split_sequences` function, creating input-output pairs for time-series prediction.

- **Machine Learning Model:**

- Uses a CNN with the following architecture:
 - * Input layer accepting sequences of shape (3, 4), where 4 features are humidity, temperature, lighting, and moisture.
 - * Two Conv1D layers (32 and 64 filters, kernel size 2, ReLU activation, ‘he_normal’ initialization, padding ‘same’).
 - * MaxPooling1D layer with pool size 1 to retain sequence information.
 - * Flatten layer to convert to 1D.
 - * Two dense layers (100 and 50 units, ReLU activation, ‘he_normal’ initialization).
 - * Output layer with 4 units (for the 4 features) and linear activation for regression.
- Compiled with Adam optimizer (learning rate 0.001), MSE loss, and MAE metric.
- Trained with early stopping (monitoring `val_loss`, patience 20, min_delta 0.0001, restoring best weights), 200 epochs, batch size 32, and 20% validation split.

- **Prediction and Integration:**

- The `prediction` function takes current feature sequences, normalizes them, and uses the trained model to forecast the next values, then inverse-transforms to original scale.
- Integrates with real-time data via serial communication, parsing sensor readings from a serial port (COM5) for temperature, humidity, light, and moisture.
- Uses MQTT communication to connect to a broker (`app.coreiot.io`) via Paho MQTT client, sending sensor data and predictions to a server (likely CoreIoT/ThingsBoard), and handling RPC requests (e.g., for controlling a switch).

This implementation supports real-time environmental monitoring and prediction, suitable for IoT applications like smart homes or agriculture, with predictions sent to cloud platforms for further analysis.

11 Incorporate YOLO for Detection

In intelligent fire monitoring systems, object-detection models like YOLOv8 provide a strong balance of speed and accuracy for real-time inference. By fine-tuning on a specialized fire detection dataset, effective EDA reveals class imbalance and image quality issues that guide augmentation and training decisions, while Roboflow's AI-assisted labeling accelerates dataset curation and ensures high-quality annotations for both bounding boxes and classification tasks.

11.1 Model Architecture

YOLOv8 is one of the latest generation of the "You Only Look Once" family, featuring an anchor-free, multi-scale Ultralytics head, state-of-the-art backbone and neck architectures, and an optimized accuracy-speed trade-off suitable for real-time detection tasks. Compared to earlier YOLO versions, it supports multiple tasks (object detection, segmentation, keypoints) and provides superior performance for fire detection applications requiring immediate response capabilities.

Starting with multi-scale detection, the modern YOLO family (from v3 through v8) adopts a set of parallel "detection heads" that each look at feature maps of different spatial resolutions—typically at strides of 32, 16, and 8 pixels. By attaching small prediction modules on top of deep, medium, and shallow layers of the backbone, these models can simultaneously spot large, medium, and tiny objects in a single forward pass.

11.2 Dataset

To train an effective object detection model, especially one that's critical like detecting fires on home, we need access to high-quality, well-annotated. For this purpose, we use the dataset hosted on [Roboflow Universe](#). Roboflow is a platform designed to streamline the dataset management process for computer vision tasks. Roboflow allows users to export datasets in a variety of formats compatible with YOLO. Another part of its strength is that it is well integrated with the previously mentioned Ultralytics.

The dataset contains a total of 2,326 images, it is focused on fire detection, specifically designed to identify and classify instances of fire and potentially related elements like smoke. The name *amenazas* (Spanish for "threats") suggest hazardous scenarios such as "fire" and "smoke" in various environments. The images are typically resized or preprocessed to a standard resolution suitable for YOLOv8 models, which is commonly 640x640 pixels. This is a default setting for many Roboflow datasets exported in YOLO format. There are 1,596 training images (70%), 521 images on the validation set used to tune hyperparameters and monitor training performance, and 219 images from the Test set (10%), reserved for final evaluation of the model's performance. The model after training using this dataset could be applied in surveillance systems, wildfire monitoring, or industrial safety applications to detect fire hazards early and trigger alerts.

11.2.1 Label Definitions

Our fire and smoke detection dataset comprises high-resolution images annotated for binary fire classification to support real-time fire monitoring in agricultural and residential environments. Data were collected under diverse lighting and environmental conditions to ensure model robustness.

The dataset focuses on fire and smoke detection where:

- Class 0 (Fire): Indicates active fire and flames, requiring immediate attention.
- Class 1 (Smoke): Indicates active smoke, or other combustion indicators requiring immediate attention.
- Undetected: Represents safe environmental conditions without fire hazards.

About the dataset used for yellow leaf detection (can be accessed here: [HLB_train](#)), it focuses on Huanglongbing (HLB) detection, a disease affecting citrus trees, often indicated by yellowing leaves. The dataset contains 1,278 images. This is based on typical sizes for Roboflow datasets in similar domains, though exact counts may vary slightly with updates. The original annotations include classes such as "HLB" (for leaves showing yellowing or other HLB symptoms) and "Healthy" (for unaffected leaves), focusing on leaf conditions in citrus trees. However, because we only need to classify the leaves as yellow or non-yellow, all healthy labelled images are now re-classified as Non-yellow, and the rest are Yellow leaves. The metadata also changed to match our use case of binary classification.

The images are resized to 640x640 pixels, a standard resolution for YOLO models on Roboflow, ensuring compatibility with training pipelines like the one in your fire detection notebook.

The detection system uses a binary classification approach:

- Class 0: **Yellow** - Represents yellow/diseased leaves
- Class 1: **Non-Yellow** - Represents healthy/green leaves

11.2.2 Exploratory Data Analysis

To improve model generalization and robustness, we applied comprehensive augmentations through Roboflow's preprocessing pipeline, including:

Table 12: Augmentation Methods Applied in the Preprocessing Pipeline for Yellow Leaf Detection

Method Category	Description
Geometric Transformations	Horizontal flips, rotations ($\pm 15^\circ$), scaling ($\pm 20\%$)
Color Space Augmentations	Brightness adjustments ($\pm 25\%$), contrast modifications ($\pm 20\%$), saturation shifts ($\pm 15\%$)
Noise and Quality Variations	Gaussian noise injection, blur effects, compression artifacts simulation
Environmental Adaptations	Lighting condition variations to simulate different times of day and weather conditions
Crop and Resize Operations	Random cropping and aspect ratio adjustments while preserving bounding-box consistency
Perspective Transformations	Random perspective shifts and shear distortions ($\pm 10^\circ$)
Motion Blur	Simulated motion effects with varying blur kernels (up to 5x5 pixels)
Edge Enhancement	Sharpening filters to emphasize leaf edges or boundaries
Color Jitter	Random hue, saturation, and value (HSV) jittering beyond standard shifts
Occlusion Simulation	Partial occlusion with random patches to mimic overlapping leaves or objects

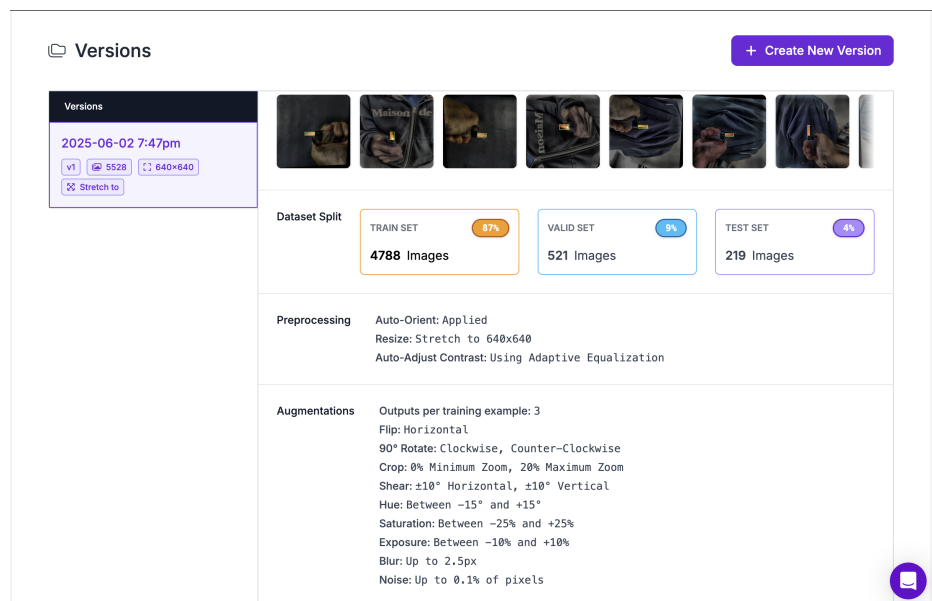


Figure 18: Fire Detection Dataset: Data Preprocessing and Augmentation on Roboflow

These augmentations significantly improved dataset diversity, increasing our effective training data from 2,336 images to nearly 5,600 images, while maintaining annotation accuracy and ensuring the model's ability to detect fire under various real-world conditions.

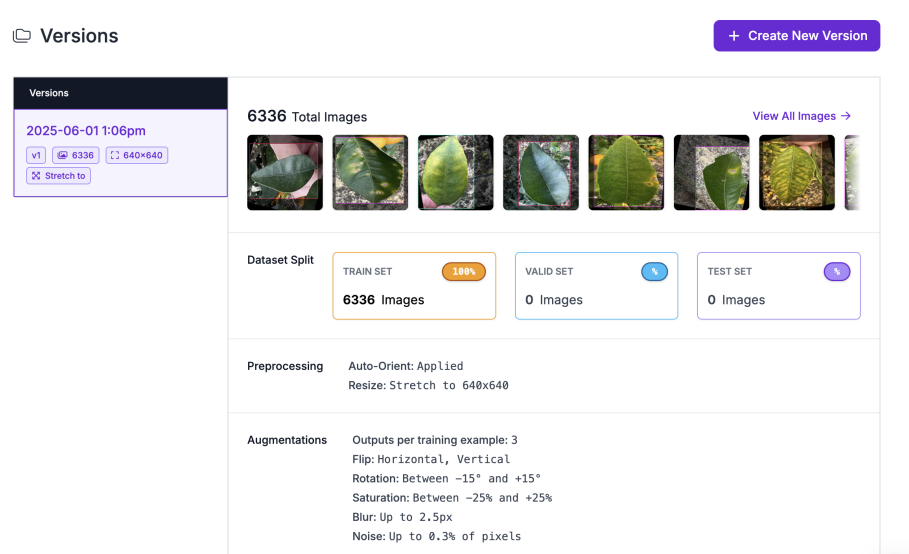


Figure 19: Yellow Leaf Dataset Preprocessing and Augmentation on Roboflow

11.3 Activity Diagram

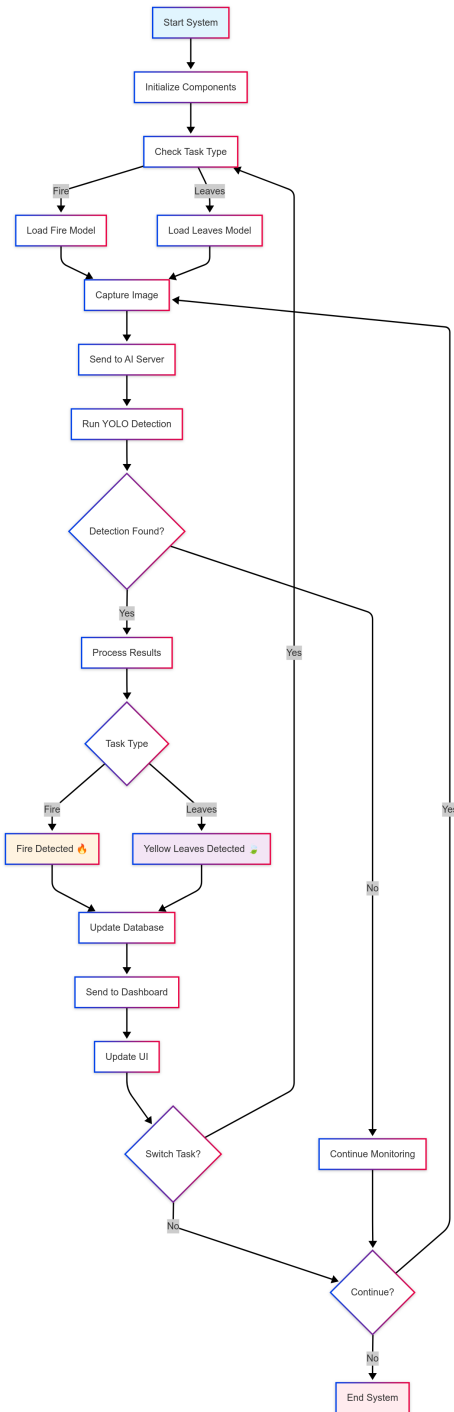


Figure 20: Activity Diagram of Fire/Leaves Detection

The provided activity diagram represents a streamlined system workflow for detecting fire or yellow leaves using an ESP32 simulator, an AI server, and a dashboard interface.

The flowchart follows a logical sequence from system initialization to task-specific detection and result processing, making it easy to understand the flow of operations. It captures images, sends them to the AI server for YOLO detection, and processes results only when detections are found, optimizing resource use. The system includes a “Continue Monitoring” loop, enabling real-time monitoring, and a “Switch Task?” decision point. After sending the frames to the model, it returns the results, which are sent back to the dashboard and update the UI, ensuring that end-users receive timely feedback, which is critical for applications like fire or plant health monitoring. The “Continue?” decision point with an “End System” option provides a clear exit condition, preventing indefinite operation when no longer needed.

11.4 Sequence Diagram

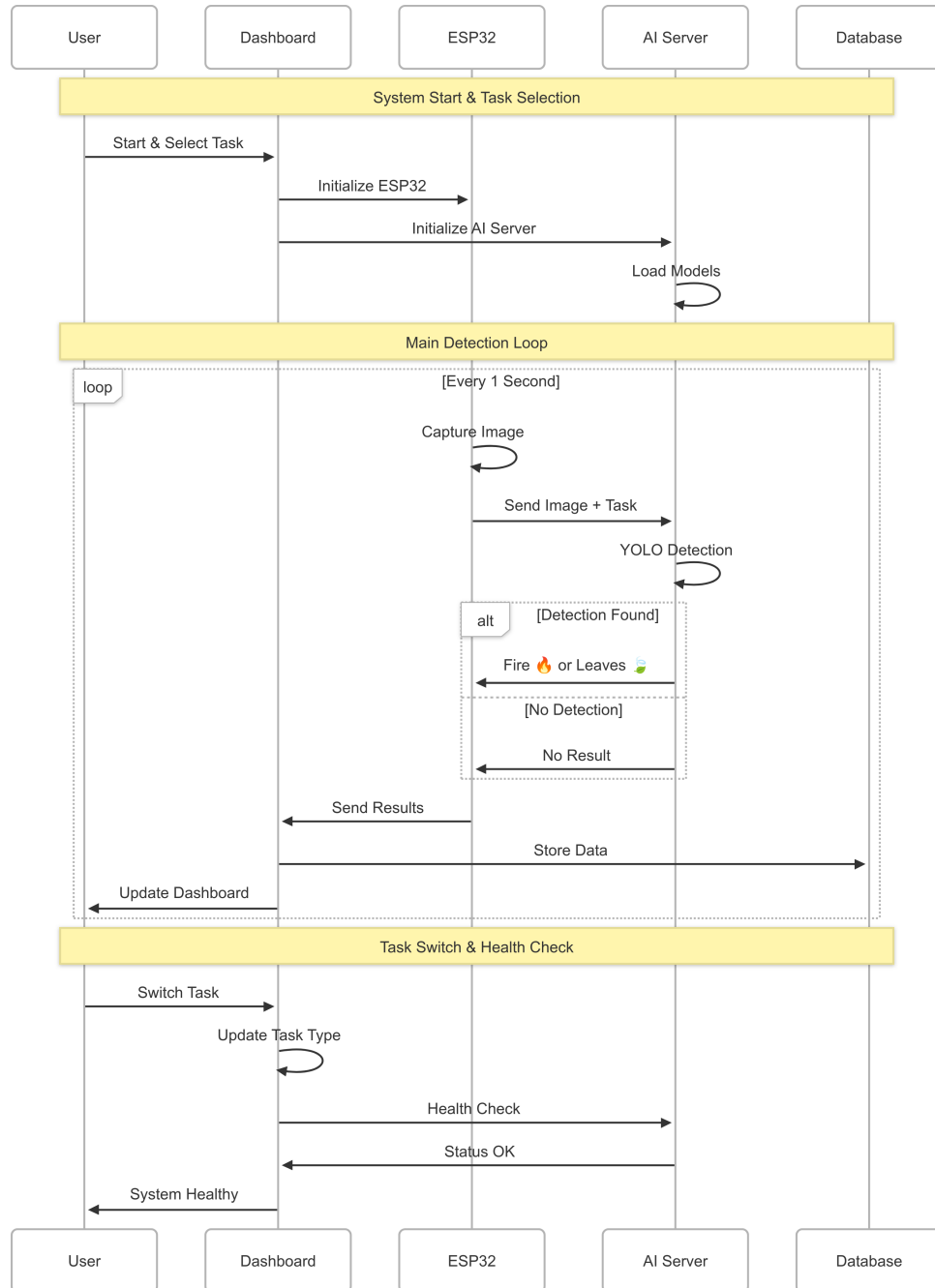


Figure 21: Sequence Diagram of Fire/Leaves Detection

The sequence diagram outlines a comprehensive workflow for a detection system designed to identify fire or yellow leaves using an ESP32 simulator, an AI server, and a dashboard interface.

To elaborate further, it describes a cyclical process involving the following components:

- ESP32 Simulator: Acts as the data acquisition unit, capturing images and determining the detection task (Fire or Yellow Leaves).
- AI Server: Performs YOLO-based inference using task-specific models.
- Dashboard: Provides a user interface to display detection results and update statistics.
- Database: Stores detection results for historical analysis and tracking.
- The system initializes components, selects a task, captures and processes images, performs detection, updates counters, and loops back for continuous monitoring. It also includes task-switching capabilities and system termination logic.

11.5 Training

The training process for the fire detection model utilized a notebook on a Kaggle environment with GPU acceleration (Tesla T4). Here we integrated the Ultralytics YOLOv8 framework, version 8.3.147, with Python 3.10.12 and PyTorch 2.5.1, running on a system with 4 CPUs, 31.4 GB RAM, and 6,358 GB of available disk space out of 8,062 GB. The dataset used was Amenazas-1, downloaded via Roboflow, containing annotations for fire detection.

The model selected was yolov8m.pt, a medium-sized YOLOv8 variant, pre-trained and downloaded with 49.7 MB. The training configuration is enclosed in the API call from `yolo`.

11.6 Evaluation

11.6.1 Result

Based on the training result, we have come up with key observations:

- Loss Curves (Training and Validation):
 - Train/Box Loss: Decreases from 2.0 to 1.4 over 20 epochs, showing the model is effectively learning to localize bounding boxes.
 - Train/Cls Loss: Drops from 2.25 to 1.0, indicating good progress in classifying objects.
 - Train/Dfl Loss: Reduces from 1.9 to 1.5, reflecting improved distribution focal loss, which helps with objectness and classification refinement.
 - Val/Box Loss, Val/Cls Loss, Val/Dfl Loss: All show a downward trend, aligning with training losses, suggesting the model generalizes well to the validation set without significant overfitting.
- Precision and Recall: Increases from 0.3 to 0.7, though it plateaus slightly toward the end, suggesting a potential limit in detecting all instances. The balance between precision and recall improves, which is critical for detecting yellow leaves where false negatives (missed diseased leaves) could be costly.

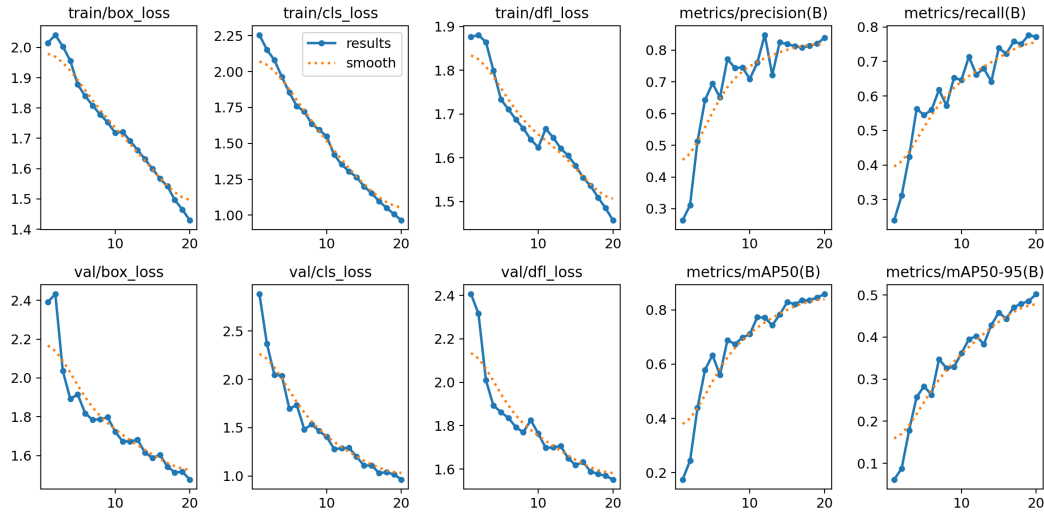


Figure 22: Result of training the Fire detection Model

- mAP Metrics:

- mAP50(B): Climbs from 0.2 to 0.8, a strong indicator of the model's ability to detect objects with an IoU threshold of 0.5, showing high accuracy.
- mAP50-95(B): Increases from 0.1 to 0.5, reflecting decent performance across various IoU thresholds (0.5 to 0.95), though it's lower than mAP50, which is expected due to the stricter evaluation.
- These metrics suggest the model is effective for practical use, especially with the high mAP50 value.

- General trends:

- Convergence: All loss curves show a consistent decline, with smoothing (orange dashed line) closely tracking raw results (blue line), indicating stable training without erratic behavior.
- Generalization: The validation losses are slightly higher than training losses but follow a similar trend, suggesting the model generalizes well to unseen data, supported by the rising mAP and recall.
- Plateauing: Toward epoch 20, some metrics (e.g., recall, mAP50-95) show signs of leveling off, which might indicate the model has reached a local optimum with the current dataset size and epochs (20). Increasing epochs or applying more aggressive augmentations could help.

11.6.2 Test set Performance

We evaluated our trained YOLOv8 model on a held-out test set comprising diverse fire scenarios and environmental conditions. The model demonstrates robust detection capabilities across various

lighting conditions, fire intensities, and background environments.



Figure 23: Sample predictions on the validation set.

Our evaluation reveals strong performance in fire detection scenarios:

- **High Confidence Detection:** The model successfully identifies active fires with confidence scores above 0.90, as demonstrated in some of the validation image where a building fire is detected with 100% confidence (class 0: fire, confidence: 1)
- **Accurate Bounding Box Localization:** Blue bounding boxes precisely encompass fire regions, indicating effective spatial localization of fire hazards. Light Blue bounding boxes indicate smoke areas. Red bounding boxes indicate ground truth fire/smoke.
- **Robust Environmental Adaptation:** Detection maintains accuracy across various backgrounds, lighting conditions, and fire intensities.

12 Design Pattern

12.1 Observer

- Description: The Observer Design Pattern is a type of behavioral pattern that establishes a one-to-many relationship between objects. When the state of one object (the subject) changes, all related objects (observers) are automatically informed and updated. This pattern mainly addresses how objects communicate and respond to changes in each other's state.
- Advantages:
 - One object will notify multiple others about changes.
 - Keep objects loosely connected.
 - Observers automatically respond to changes in the subject's state.
 - Easily add or remove observers without changing the main subject.
 - Deal with event systems that require various components to react without direct connections.
- Disadvantages:
 - Is complicated for simple relationship between objects and don't require notifications.
 - Many observers can lead to overhead during updates.
 - In case the subject and observers are tightly coupled, it breaks the purpose of decoupling.
 - Unpredictable order of notifications may be a problems when the notification order is crucial.
- Applications: Used in front-end to get real-data.

12.2 Module

- Description: The Module pattern is a design approach that enables the creation of objects with private variables and functions that are inaccessible from the outside. It encapsulates both public and private members, helping to safeguard internal details and prevent conflicts in the global scope or with other developers' code.
- Advantages:
 - Offers a method to contain private variables and functions, keeping them hidden from external access.
 - Assists in keeping the global scope tidy and minimizes the likelihood of name conflicts.
 - Simple to implement and helps organizing code by clearly separating responsibilities.
- Disadvantages:

- Testing private functions and variables can be challenging, as they are not directly accessible.
 - It lacks support for inheritance, making it impossible to create new objects that extend the functionality of the original.
- Applications: Used in front-end that makes it easily to implement, manage, maintain and extend in the future.

13 Main Module

13.0.1 Module 1: Receive and display data from devices

This module collects environmental data (e.g. temperature, humidity, soil moisture, light) from IoT sensors installed on the farm.

Data is transmitted via serial protocol to the edge device (laptop), after being processed at the edge device, they will be pushed to coreIOT via MQTT method

How it works:

- Each sensor will read data, then the data is gathered into 1 message and transmitted to the edge device.
- then edge device will send this value to coreIOT via MQTT method, ensuring low-latency updates.
- The front-end will get the values from coreIOT through rest API
- The front-end subscribes to backend JSon and updates UI components like:
- Line charts for trends.
- Status cards for real-time readings.
- Colored indicators based on sensor health.

13.0.2 Module 2: Check received data exceeds allowed threshold

Ensures that incoming sensor values are compared against their configured thresholds.

If a value exceeds normal bounds (e.g., moisture $\geq 20\%$), the system assigns a status like "warning" or "critical".

How it works:

- Every sensor has min, max, critical thresholds.
- Based on the status, the UI may change color, and an alert may be generated.

Example: A soil sensor reports value = 10. Thresholds: min = 15, critical = 20 → The system assigns "warning" and triggers a yellow warning in the UI.

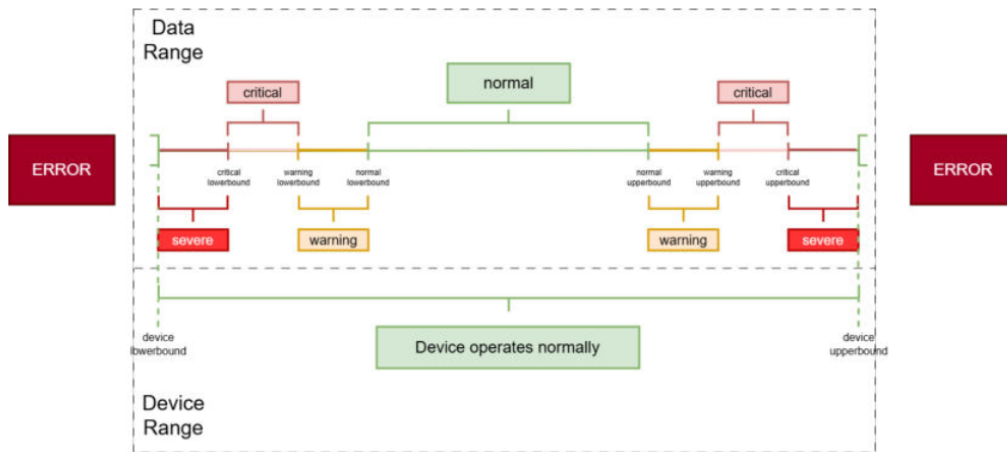


Figure 24: Threshold range

13.0.3 Module 3: Device control

Allows manual or automatically control of farm devices like water pumps, fan motor and RGB grow lights.

How it works:

- Manual Mode: Users click buttons in the Control Panel to toggle devices on/off.
- Scheduled Mode: Devices activate based on pre-configured times (e.g., 6 AM to 8 AM).
- Auto Mode: Device control is triggered by sensor values (e.g., pump turns on if soil is dry).

Communication Flow:

- The Web App sends a command (via restAPI) with action
- The server receives the command and relays it to the hardware (e.g., ESP32) via MQTT.
- The hardware executes the action and may return a confirmation via MQTT.

13.1 Module 4: Record activities

Logs all device control activities, sensor triggers, and status transitions.

Helps with system diagnostics, auditing, and future improvements (e.g., AI prediction).

How it works:

- Every action—such as turning on a pump, changing a light's color, or a sensor crossing a threshold—is recorded with a timestamp.
- This information is stored in a backend database (Backend Json).
- The front-end provides a UI to review logs, filter by device or type, and visualize trends.

13.1.1 Module 5: Web application

The front-end provides a responsive interface for monitoring, control, scheduling, notifications, and reminders.

- Dashboard: Displays live data cards, charts, and trends.
- Control Panel: Lets users toggle pumps/fan/lights and set schedules/modes.
- Reminders: Displays and schedules user-defined events like watering reminders, fan on reminders.
- Notifications: Displays real-time alerts when thresholds are breached.

14 UI Description

14.1 Core Dashboard

The CoreIOT Dashboard serves as the central user interface of the environmental monitoring system, enabling real-time visualization, control, and analysis of sensor data deployed throughout the greenhouse. Designed with user-centric interaction in mind, the dashboard facilitates informed decision-making, rapid response to environmental changes, and ongoing optimization of agricultural conditions.

14.1.1 System Objectives

The CoreIOT Dashboard was developed with the overarching goal of enabling intelligent environmental monitoring and control in greenhouse or agricultural settings. Its key objectives include the following:

- **Centralized Monitoring:** To provide an intuitive and unified interface for visualizing sensor data such as temperature, humidity, light intensity, and soil moisture. The dashboard aggregates real-time measurements from distributed sensors, helping users easily observe environmental changes without the need for manual field inspections.
- **Real-Time Alerts:** To support automated threshold-based alerting mechanisms. When any monitored parameter exceeds the defined warning or critical range, the system immediately triggers visual or audible alerts on the dashboard, enabling users to take timely corrective action.
- **Historical Data and Analytics:** To serve as a platform for recording and analyzing historical data. Users can review trends over time, correlate environmental factors with crop performance, and make informed decisions about climate management strategies or energy usage.
- **Interactive Device Control:** To enable direct interaction with actuators through on-screen buttons and controls embedded in the dashboard. Users can remotely turn on or off connected devices such as LED grow lights (to adjust lighting conditions), ventilation fans (to regulate air circulation and temperature), and water pumps (to manage irrigation and soil moisture levels). These control features are designed to be intuitive and responsive, allowing real-time actuation with status feedback, thereby closing the loop between sensing and action.
- **Scalability and Automation Integration:** To lay the groundwork for integrating rule-based automation and artificial intelligence in future iterations. This includes the potential to automate device control based on environmental triggers, time schedules, or predictive models, reducing manual intervention and increasing operational efficiency.

14.1.2 Main Dashboard Overview

The main dashboard interface of the CoreIOT system is designed to deliver a comprehensive and real-time view of the current environmental conditions along with manual control capabilities for essential actuators. As shown in the figure, the interface is divided into several distinct functional sections:

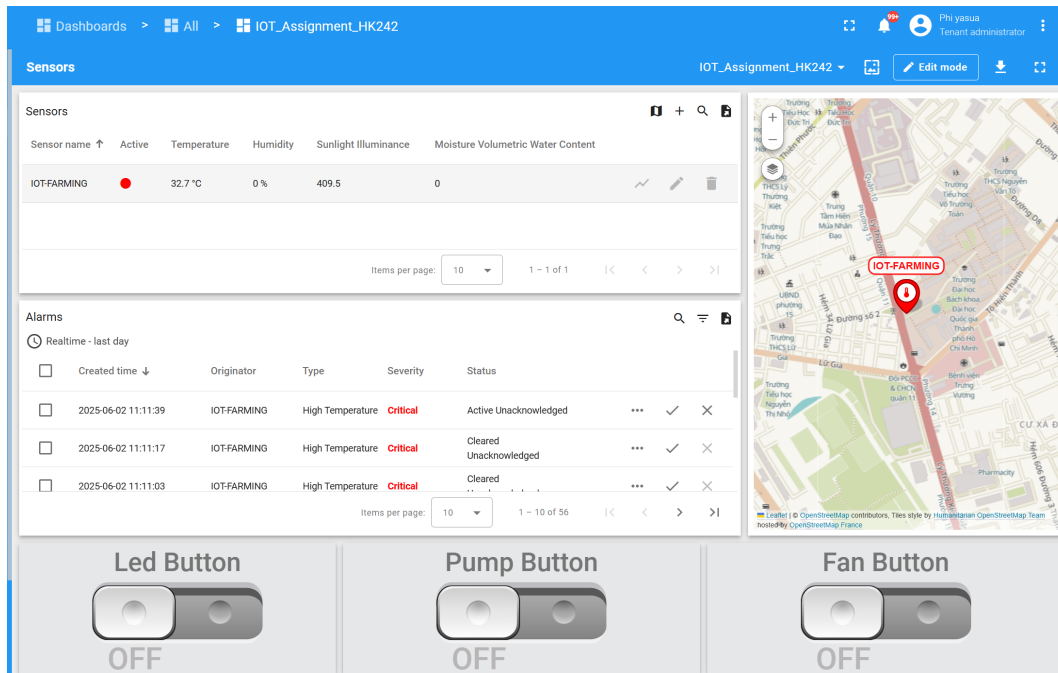


Figure 25: Main dashboard

- **Sensor Summary Panel (Top-Center):** This section provides a tabular summary of real-time data collected from the deployed sensors. It includes key parameters such as:
 - Temperature (e.g., 32.7°C),
 - Humidity (e.g., 0%),
 - Sunlight Illuminance (e.g., 409.5 lux),
 - Moisture Volumetric Water Content (e.g., 0%).

Each sensor is listed by name (e.g., IOT-FARMING), and its active status is indicated with a colored dot for quick diagnostics. Moreover, by clicking on a specific sensor entry, the user can access a detailed view that includes time-series charts, historical trends, and performance analytics specific to that sensor device. This feature allows for deeper investigation into fluctuations and long-term environmental patterns, which supports decision-making and troubleshooting. .

Sensors								
Sensor name ↑	Active	Temperature	Humidity	Sunlight Illuminance	Moisture Volumetric Water Content			
IOT-FARMING	●	32.7 °C	0 %	409.5	0			

Items per page: 10 1 – 1 of 1

Figure 26: Sensor Summary Panel

- **Alarm Panel (Center-Bottom):** This section shows a real-time feed of alarms and alerts generated based on threshold breaches. For example, the dashboard indicates several **High Temperature** alarms with a severity marked as **Critical**, along with status updates such as **Active Unacknowledged** or **Cleared**. Users can acknowledge or clear alarms directly from this panel to maintain situational awareness.

Alarms								
	Realtime - last day							
☐	Created time ↓	Originator	Type	Severity	Status			
☐	2025-06-02 11:11:39	IOT-FARMING	High Temperature	Critical	Active Unacknowledged	...	✓	✕
☐	2025-06-02 11:11:17	IOT-FARMING	High Temperature	Critical	Cleared Unacknowledged	...	✓	✕
☐	2025-06-02 11:11:03	IOT-FARMING	High Temperature	Critical	Cleared	...	✓	✕

Items per page: 10 1 – 10 of 56

Figure 27: Alarm Panel

- **Notification Popup (Top-Right Corner):** In addition to the centralized alarm table, the system also features a real-time notification popup component designed to enhance user awareness and response speed. This popup displays a chronological list of active and resolved alerts in a compact format, including the following details:
 - Type of alarm (e.g., **High Temperature**, **High Humidity**),
 - Severity level (e.g., **Critical**, highlighted in red),
 - Source of the alert (e.g., device name **IOT-FARMING**),
 - Timestamp (e.g., “18 hr ago”),
 - Status (e.g., **cleared**, **new**, or **unacknowledged**).

Each notification item is accompanied by a bell icon and a check mark icon for manual acknowledgment. New critical alerts are outlined in red to draw immediate attention. This

popup interface is automatically triggered upon the generation of any new alarm and remains accessible for quick review. A “Mark all as read” option is available to manage visibility, while the “View all” link at the bottom redirects the user to the full alarm history page.

This lightweight alerting mechanism ensures that users are promptly informed about significant events, even when they are not actively viewing the main alarm panel.

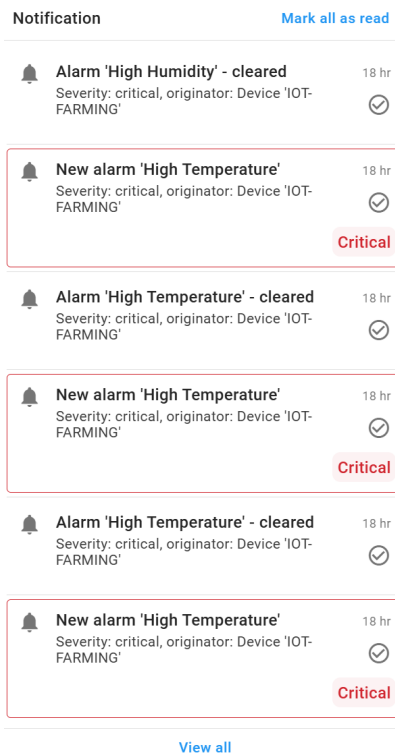


Figure 28: Alarm Panel

- **Geolocation Panel (Right):** The system integrates OpenStreetMap to display the geographic location of the sensor node on an interactive map interface. The sensor **IOT-FARMING** is represented with a prominent red icon, and any active alerts (such as critical temperature thresholds) are immediately visible through a red alert badge overlay.

When the user clicks on the icon, a popup appears containing essential real-time data including temperature (e.g., 32.7°C) and humidity (though in this image the humidity is not available or not yet initialized). A clickable link labeled **Sensor details** provides navigation to a more detailed view of the device’s historical data and analytics.

This geolocation feature supports intuitive spatial awareness for users managing multiple sensor nodes across distributed zones. It enables quick pinpointing of environmental anomalies and helps with decision-making related to location-specific interventions such as targeted irrigation or localized cooling solutions.

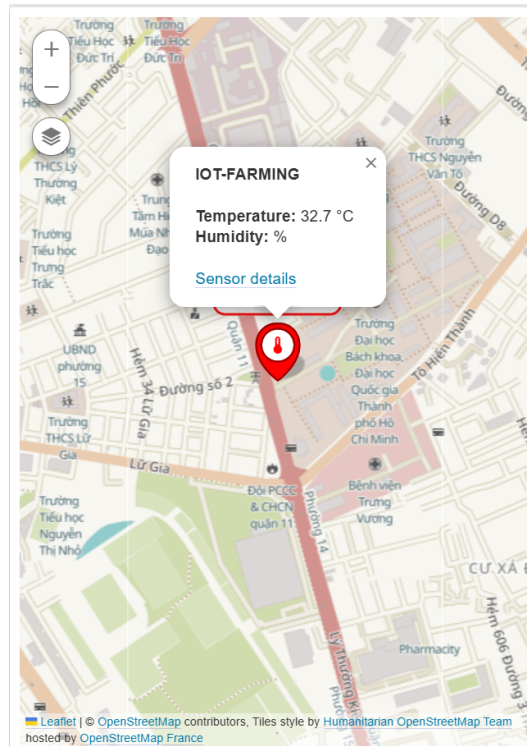


Figure 29: Geolocation Panel

- **Device Control Buttons (Bottom):** Three toggle switches labeled LED Button, Pump Button, and Fan Button allow the user to manually control actuators connected to the system. Each button shows the current state (OFF) and can be toggled to perform operations such as:
 - Turning on/off the grow light (LED),
 - Activating/deactivating the irrigation pump,
 - Enabling/disabling ventilation fans.

These controls empower users to respond to sensor data dynamically and immediately from the dashboard itself.

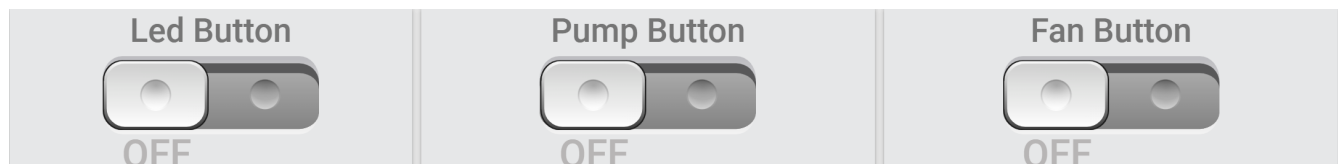


Figure 30: Device Control Buttons

14.1.3 Real-Time Environmental Monitoring Subdashboard

The subdashboard component of the IOT-FARMING platform serves as a comprehensive interface for monitoring real-time environmental data collected from distributed sensor nodes deployed in agricultural fields. This subdashboard is designed to support data-driven decision-making by presenting critical sensor readings in a highly visual, modular, and interactive layout. Each panel within the interface corresponds to a specific environmental parameter, allowing users to observe trends, identify anomalies, and manage alerts effectively.

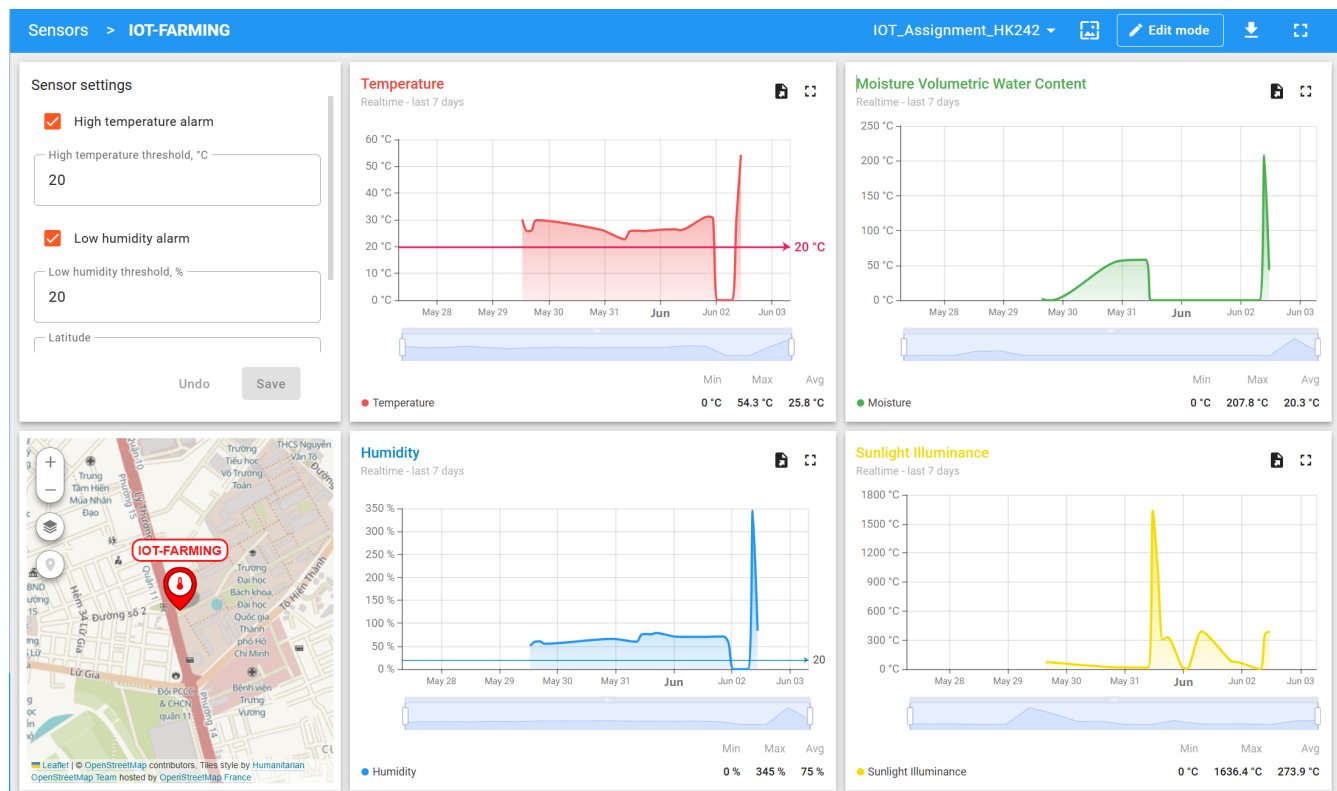


Figure 31: Subdashboard

- **Temperature Panel:**

This panel visualizes the historical temperature data collected over the past seven days through a dynamic time-series line graph. The red-colored plot intuitively represents the degree of temperature variation, enabling users to identify both gradual trends and sudden spikes. An important feature of this panel is the integration of a user-defined threshold, currently set at 20°C, which acts as a benchmark for triggering alerts. When the temperature exceeds this threshold, a red shaded region appears on the chart, drawing immediate attention to potentially critical conditions.

The maximum temperature value recorded in the observed period was 56.2°C, which is significantly above typical agricultural tolerances and could point to an extreme heat event or a sensor malfunction. The average temperature during the week stands at 27.5°C, suggesting

sustained moderate heat across several days. By monitoring this panel, stakeholders can better evaluate the potential impacts of thermal stress on crops, soil evaporation rates, and the need for shading or irrigation interventions.

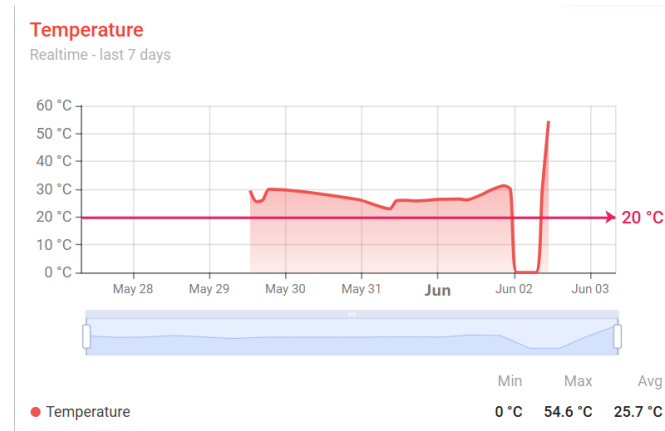


Figure 32: Temperature Panel

- Moisture Volumetric Water Content Panel:**

The moisture panel provides an essential view into the volumetric water content of the soil, which is a fundamental variable in precision irrigation planning. Represented as a green-filled area chart, it offers insights into soil water availability over time and highlights fluctuations that may affect crop hydration.

The chart shows a significant upward spike on June 2, with the moisture reading reaching a peak of 207.8%. While this value is unusually high—possibly due to excessive rainfall or sensor calibration drift—it flags the importance of soil drainage and aeration. Throughout the week, the average soil moisture remains at 20.3%, which is within an acceptable range for most crops. Through this visualization, farmers can optimize watering schedules, reduce water waste, and avoid problems such as root rot or drought stress.

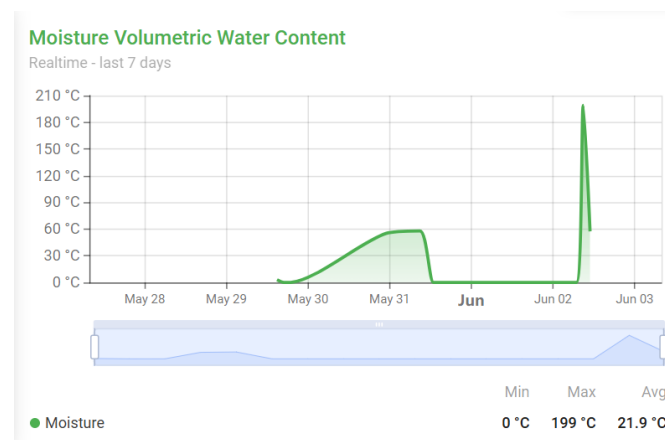


Figure 33: Moisture Volumetric Water Content Panel

• Humidity Panel:

This panel captures atmospheric humidity levels, an important environmental factor that influences transpiration, pest behavior, and plant metabolism. The plotted data shows a relatively consistent range throughout the first half of the week, followed by a sudden increase toward the end.

The maximum humidity reading reached an extreme of **294%**, which exceeds physiological norms and could indicate sensor misreporting or extreme condensation. Nevertheless, the average remains at **76%**, which is conducive to most crop types, provided proper ventilation is maintained. The system uses a low-humidity threshold of **20%** to issue early warnings for drought-like conditions. Maintaining this metric helps ensure adequate photosynthetic efficiency, disease control, and overall crop vitality.

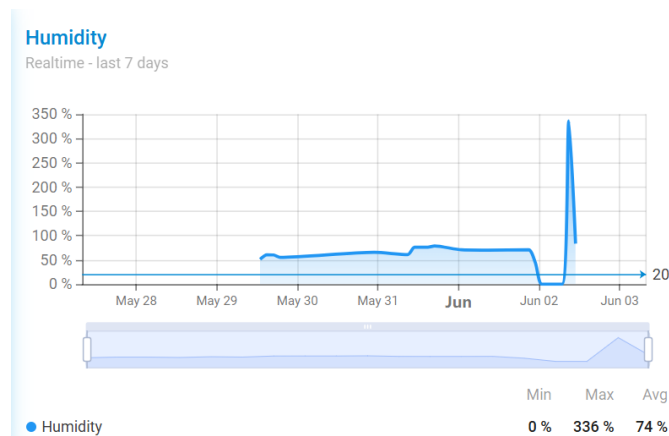


Figure 34: Humidity Panel

• Sunlight Illuminance Panel:

The sunlight illuminance panel reports on the solar energy reaching the crops, a key determinant of photosynthetic activity and plant growth. The data is plotted as a yellow line across the chart and allows for immediate interpretation of light exposure patterns.

The peak illuminance recorded was **1636.4 lux**, with an average value of **273.9 lux**. These readings suggest dynamic light conditions with periods of both intense exposure and shading, likely due to cloud cover or diurnal variation. Tracking this metric helps determine whether plants receive adequate light energy and informs the use of shading nets or artificial grow lights. Moreover, sunlight variability is critical for adjusting watering patterns, as higher radiation accelerates soil moisture loss and evapotranspiration.

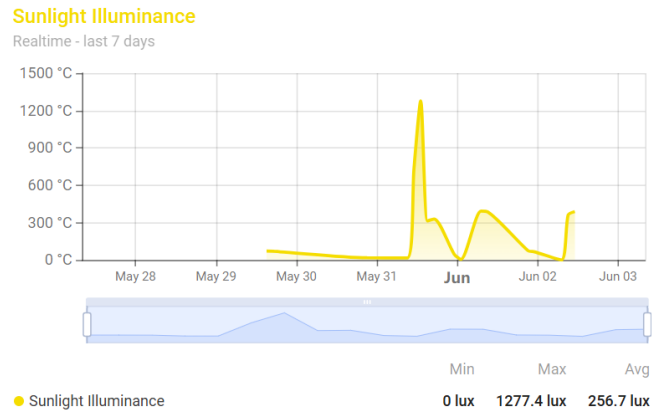


Figure 35: Sunlight Illuminance Panel

- **Geolocation Panel:**

This map-based panel provides a spatial interface by integrating OpenStreetMap into the system, displaying the real-world location of the deployed sensor node. The sensor, tagged as IOT-FARMING, is marked with a distinct red icon that visually represents its operational state. When certain environmental thresholds are breached (e.g., extreme temperature or low humidity), the icon is overlaid with an alert badge.

Clicking the icon opens a popup with live sensor data, such as current temperature or humidity, and a hyperlink labeled **Sensor details** that redirects the user to a detailed diagnostics dashboard. This feature enhances geographical awareness, especially useful for large-scale farms or sensor networks deployed across varying microclimates. It facilitates localized analysis and supports decisions such as targeted irrigation or zonal pest control.

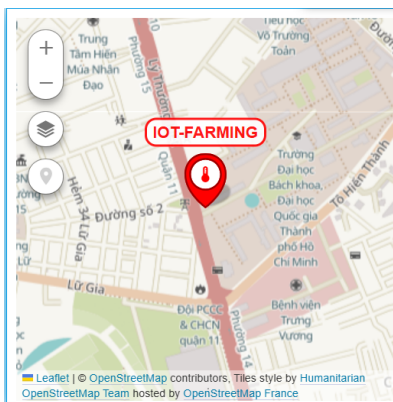


Figure 36: Geolocation Panel

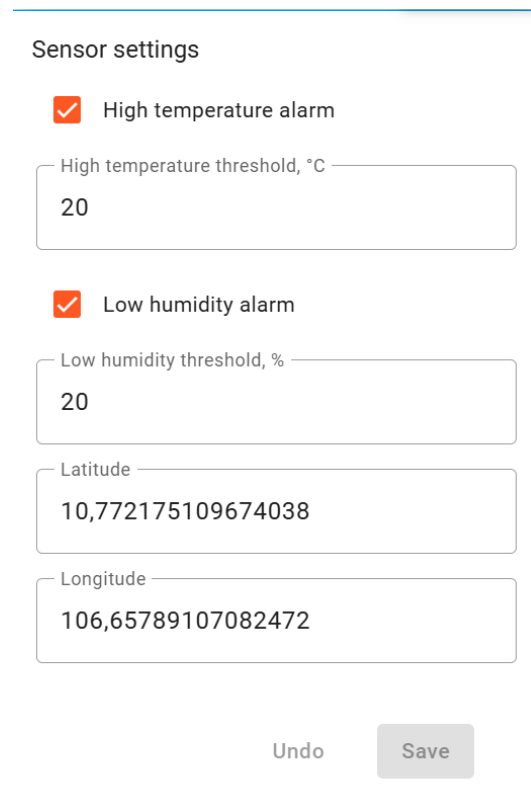
- **Sensor Settings Panel:**

This panel serves as the configuration hub where users can fine-tune environmental alarm

thresholds and define the geographic positioning of the sensor node. It provides an interactive form with clearly labeled input fields and toggleable options for environmental parameters.

- *High Temperature Alarm:* A checkbox allows the user to activate or deactivate alerts for elevated temperatures. In this case, the alarm is enabled, and the defined threshold is set at 20°C, ensuring that farmers are notified promptly when heat stress becomes a concern.
- *Low Humidity Alarm:* Similarly, this feature is enabled to detect critically low air humidity levels. The threshold is currently set to 20%, making the system responsive to drought risks or greenhouse dryness.
- *Geolocation Coordinates:* Two numerical fields allow manual input of latitude and longitude. The current values, 10.772175109674038 (latitude) and 106.65789107082472 (longitude), correspond to a deployment site in Ho Chi Minh City, Vietnam. These coordinates enable precise mapping of the sensor's data and enhance interoperability with GIS tools and mapping APIs.

The interface also includes **Undo** and **Save** buttons, supporting user experimentation and real-time control. Together, these features ensure that sensor deployment is not only reactive but also customizable and adaptive to local agronomic conditions.



The image shows a 'Sensor settings' panel with a light blue border. It contains two checked checkboxes: 'High temperature alarm' and 'Low humidity alarm'. Below each checkbox is a text input field for the threshold value, both set to '20'. At the bottom, there are two more text input fields for 'Latitude' (10,772175109674038) and 'Longitude' (106,65789107082472). At the very bottom of the panel are 'Undo' and 'Save' buttons.

Setting	Value
High temperature alarm	20
Low humidity alarm	20
Latitude	10,772175109674038
Longitude	106,65789107082472

Figure 37: Sensor Settings Panel

14.1.4 CoreIOT Dashboard Summary

The integrated dashboard system—comprising both the **Main Dashboard** and the **Subdashboard**—serves as a centralized platform for real-time monitoring, analysis, and management of critical environmental data within smart farming operations.

The **Main Dashboard** provides an overarching overview of sensor network status, including summary alerts, aggregated trends, and system health indicators. It acts as the primary point of control, offering users a high-level understanding of farm-wide conditions and enabling rapid navigation to detailed data views. Its design prioritizes responsiveness and situational awareness, allowing for quick identification of abnormal patterns such as heat stress, irrigation anomalies, or sensor failures across multiple zones.

The **Subdashboard**, in contrast, delivers a more granular and sensor-specific visualization interface. It consolidates core environmental parameters—namely temperature, humidity, soil moisture content, and sunlight illuminance—into interactive charts spanning the past seven days. Each panel not only displays historical trends but also incorporates user-defined thresholds to trigger alerts and inform timely interventions. The inclusion of a geolocation map (via OpenStreetMap) enhances spatial context by pinpointing sensor nodes and enabling site-specific data interpretation. Additionally, the sensor settings panel allows users to configure environmental alarms and assign precise GPS coordinates to each node, further improving system flexibility and field calibration.

Together, the main and subdashboards create a robust, scalable, and intuitive ecosystem for data-driven agricultural management. By enabling continuous environmental tracking, anomaly detection, and customizable response mechanisms, the dashboard system supports precision agriculture practices that enhance productivity, conserve resources, and promote sustainability.

14.2 Project Web App

The project website functions as the central hub of the environmental monitoring system, offering users an intuitive and interactive platform to access all relevant data and functionalities. It plays a critical role in the overall system architecture by enabling real-time visualization, control, and analysis of environmental sensor data collected from various locations within the greenhouse.

Through a modern and responsive user interface, the website presents dynamic charts, dashboards, and control panels that allow users to monitor temperature, humidity, soil moisture, light intensity, and other environmental parameters with high precision. These data visualizations are updated in real-time, allowing users to immediately detect anomalies, trends, or sudden changes in greenhouse conditions.

In addition to passive monitoring, the website empowers users with active control capabilities. Depending on user roles and permissions, individuals can configure threshold values for specific sensors, activate or deactivate actuators (such as ventilation fans, irrigation systems, or lighting), and receive timely alerts or notifications when certain parameters exceed acceptable ranges.

Designed with a strong focus on user-centric interaction, the website ensures accessibility and ease of use for all levels of users, from farmers and agricultural technicians to researchers and system administrators. Features such as user authentication, personalized dashboards, and device management are implemented to tailor the experience to each user's specific needs.

Ultimately, the project website serves not only as a control and monitoring interface but also as a decision-support tool. By aggregating sensor data, providing insights through visualization, and enabling immediate response actions, the website contributes significantly to optimizing greenhouse conditions, improving crop yield, and supporting sustainable agricultural practices.

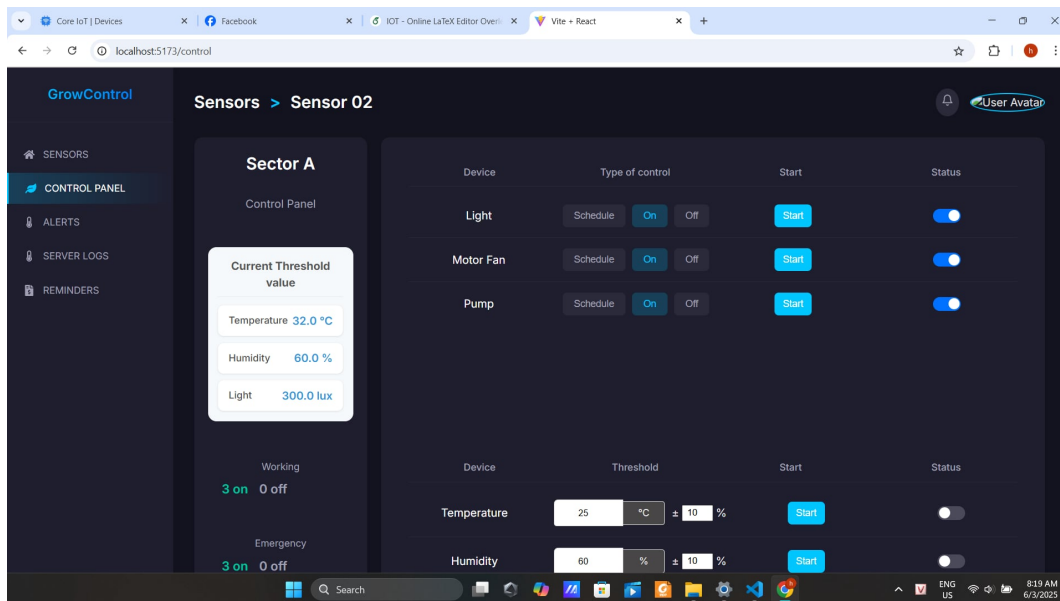


Figure 38: Website Overview

14.2.1 User Authentication: Login and Signup

The website integrates a secure and user-friendly authentication system to manage access to the environmental monitoring platform. This system includes both **signup** and **login** functionalities, ensuring that only authorized users can view, analyze, and interact with greenhouse sensor data.

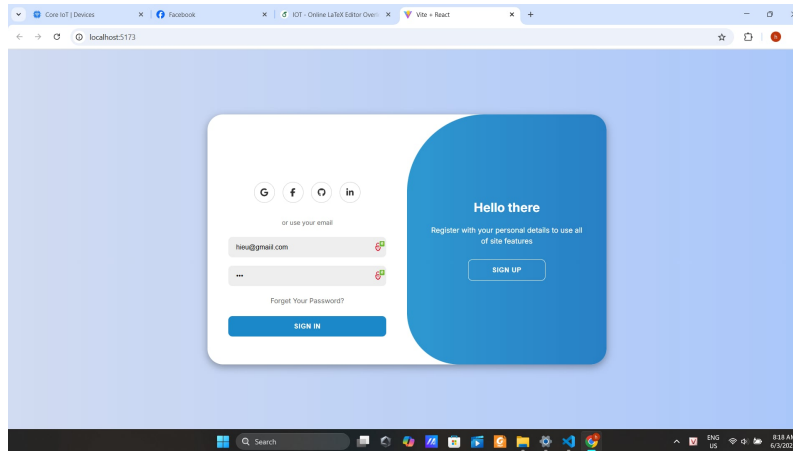


Figure 39: Signup interface for new users on the website

The **signup** process allows new users to create an account by providing essential credentials such as a unique username, a valid email address, and a secure password. This information is validated and securely stored using modern encryption techniques to prevent unauthorized access or data breaches. The system may also include password strength checks and confirmation fields to enhance user security from the point of registration.

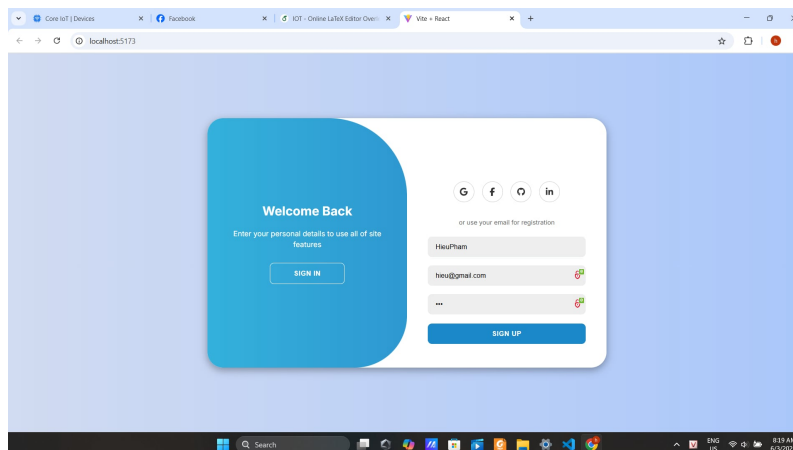


Figure 40: Login interface for registered users to access the monitoring system

Once registered, users can access the system via the **login** interface. The login process requires the correct input of previously registered credentials (email and password), which are then verified against encrypted records stored in the backend. Successful login grants access to a personalized

user dashboard, which displays real-time environmental sensor data and allows configuration of environmental thresholds tailored to the user's role or needs.

This authentication system not only safeguards sensitive data by preventing unauthorized access, but also enhances the user experience by enabling role-based access control. For instance, system administrators may have privileges to manage sensor settings or view all data streams, whereas regular users may only access data relevant to their assigned zones. Such differentiation allows for scalable and secure deployment of the platform in larger greenhouse environments.

Furthermore, features such as session management, logout functionality, and potential integration with third-party authentication providers (e.g., Google or Github accounts) can further extend the flexibility and security of the system.

14.2.2 Control Panel and Threshold Settings

The GrowControl system provides a comprehensive interface for managing environmental conditions in a controlled agricultural setting. A key component of this interface is the **Control Panel**, which allows users to monitor and manage devices in real-time. The Control Panel displays the status of devices such as the Light, Motor Fan, and Pump, each with options for scheduling or manual operation. For instance, users can toggle between “Schedule” and “On/Off” modes, and initiate immediate operation using the “Start” button. The panel also indicates the operational status of devices, with a summary showing that three devices are currently active and none are in an emergency state.

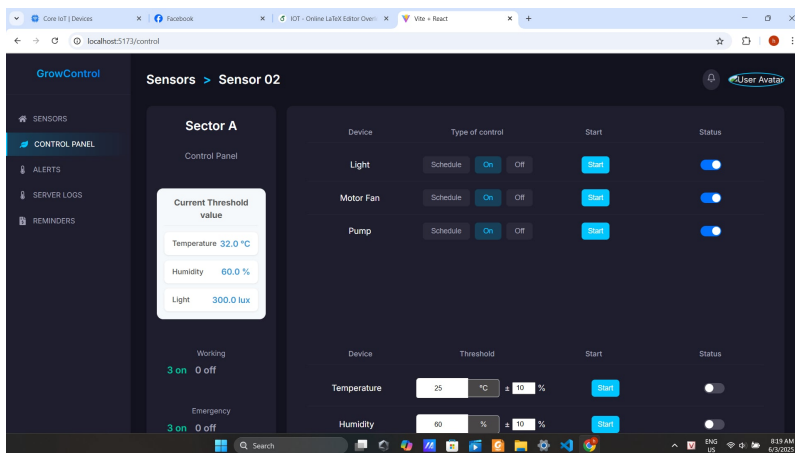


Figure 41: Control Panel interface showing device status and control options

Another critical feature is the **Threshold Settings**, which enable users to set desired environmental parameters for optimal plant growth. The interface allows configuration of thresholds for temperature, humidity, and light intensity, with current values displayed alongside the set thresholds. For example, the system shows a current temperature of 32.0°C against a threshold of 25°C $\pm$ 10%, humidity at 60.0% matching its threshold of 60% $\pm$ 10%, and light intensity at 300.0 lux against a threshold of 500 lux $\pm$ 10%. These settings ensure that the system can automatically

adjust environmental conditions to maintain them within the specified ranges, triggering alerts or device actions if thresholds are exceeded.

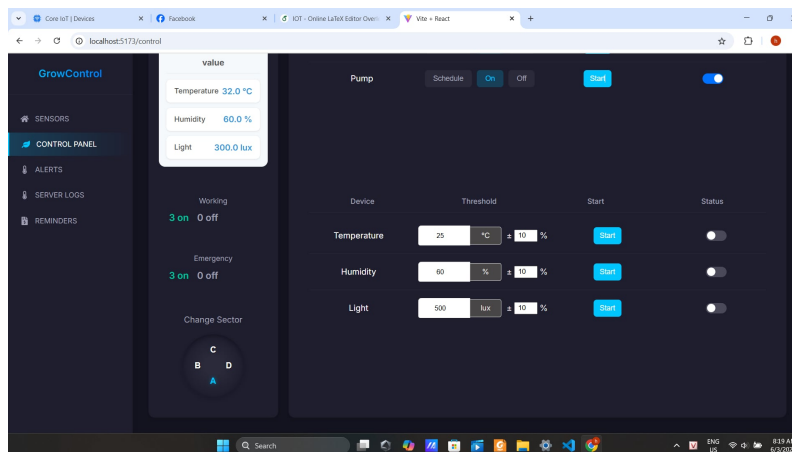


Figure 42: Threshold Settings interface displaying environmental parameters

14.2.3 Alerts and Receiving Notifications

The GrowControl system incorporates a robust alerting mechanism to ensure timely awareness of environmental conditions that deviate from predefined thresholds, a critical aspect for maintaining optimal growth environments. This subsection delves into the alert system, its categorization, and the process of receiving notifications, providing users with the tools to monitor and respond effectively.

The **Alerts** section of the interface serves as a centralized hub for tracking system status, displaying real-time notifications based on sensor data. Alerts are classified into two primary categories: *Critical* and *Warning*, each distinguished by severity and visual indicators. Critical alerts, marked with a red circle and the label “critical,” signify immediate concerns that require urgent attention. For instance, the system has recently logged multiple critical alerts, including instances where the temperature has exceeded its threshold, recorded at 11:04:10 and 11:04:10 on June 2, 2025, and a light intensity threshold breach at 10:55:48 on the same date. These alerts are designed to prompt immediate action, such as adjusting device settings via the Control Panel to mitigate potential damage to the controlled environment.

In contrast, *Warning* alerts, indicated by a yellow triangle, serve as preemptive notifications when environmental parameters approach but have not yet crossed their thresholds. The interface currently displays several warning alerts, such as temperature nearing the threshold at 10:48:06, humidity nearing the threshold at 10:47:41, and light nearing the threshold at 10:43:12, all logged on June 2, 2025. These warnings provide users with an early opportunity to intervene, preventing the escalation to critical states. The alert log includes timestamps for each event, enabling users to track the progression of conditions over time and analyze patterns that may inform future threshold adjustments.

Receiving notifications is a seamless process integrated into the GrowControl platform. Upon detection of a threshold violation or nearing condition, the system automatically generates an

alert and logs it within the Alerts section. Users can view all alerts by selecting the “All” filter or narrow down to specific categories using the “Warning” or “Critical” filters. Each alert entry includes a concise description—e.g., “Temperature has reached the threshold” or “Humidity is near the threshold”—alongside the exact time of occurrence. This detailed logging facilitates historical analysis and troubleshooting. Additionally, the interface provides a trash bin icon, allowing users to clear resolved alerts, ensuring the log remains focused on current issues.

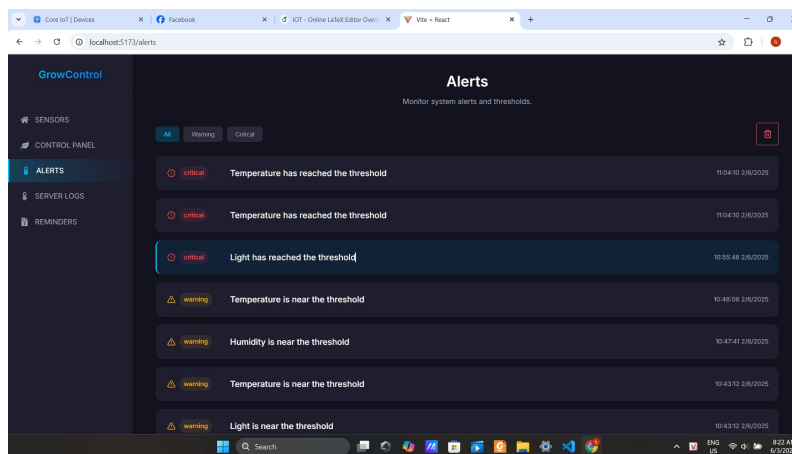


Figure 43: Alerts interface displaying critical and warning notifications with timestamps

The notification system is designed to enhance user responsiveness, with the potential for future integration of real-time push notifications or email alerts, depending on system configuration. Currently, the manual review of the Alerts section is sufficient for monitoring, but the clear categorization and timestamping ensure that users can prioritize their actions effectively. For example, the repeated critical temperature alerts suggest a need to review the cooling system’s performance, while the warning alerts indicate a gradual trend that might require preemptive threshold recalibration. This comprehensive alert and notification framework underscores GrowControl’s commitment to proactive environmental management, empowering users with the information needed to maintain ideal conditions for agricultural success.

14.2.4 Add Notes

The GrowControl system includes a robust note-taking feature under the **Reminders** section, designed to help users manage tasks and schedule activities essential for maintaining optimal environmental conditions. This subsection explores the *Add Notes* functionality, detailing how users can create, schedule, and track tasks within the system.

The **Add Notes** interface provides a streamlined form for users to input task details, ensuring that critical activities are documented and scheduled appropriately. When a user initiates the process by clicking the “Add New Note” button, a modal window appears with several fields to capture essential information. The first field allows users to enter a brief description of the task, such as “Water plant,” as shown in the interface. This description serves as a quick reference for the task’s purpose, making it easy to identify in the notes list.

Following the description, users can specify the *Date* and *Time* for the task. The date field uses a calendar picker, defaulting to the current date but allowing selection of future dates, such as “13/06/2025” in the example. The time field, formatted in a 24-hour clock, enables precise scheduling, with the example showing “08:26: SA” (indicating 08:26 AM). This scheduling capability ensures that tasks are planned at specific times, aligning with the operational needs of the controlled environment.

Another key field is the *Status* dropdown, which allows users to categorize the note’s progress. Options include “Planned,” “Completed,” and potentially others depending on system configuration. In the example, the status is set to “Planned,” indicating that the task is scheduled but not yet executed. This feature helps users track the lifecycle of tasks, from planning to completion, ensuring that no critical activity is overlooked.

Once all fields are filled, users can either save the note by clicking the green “Add Note” button or discard the entry using the “Cancel” button. Upon saving, the note is added to the Reminders list, where it appears with its status, description, date, and time. The interface also provides filters such as “All,” “Planned,” and “Completed” to organize notes, along with icons for adding, editing, or deleting notes, enhancing task management efficiency.

The Add Notes feature is integral to the GrowControl system, enabling users to maintain a structured schedule for routine tasks like watering plants or adjusting thresholds. For instance, the note “Fix the threshold” with a “Planned” status indicates an upcoming adjustment to environmental parameters, while “Turn on the pump” scheduled for the same date ensures timely irrigation. This functionality not only improves operational efficiency but also provides a historical record of activities, which can be reviewed to optimize future task scheduling and system performance.

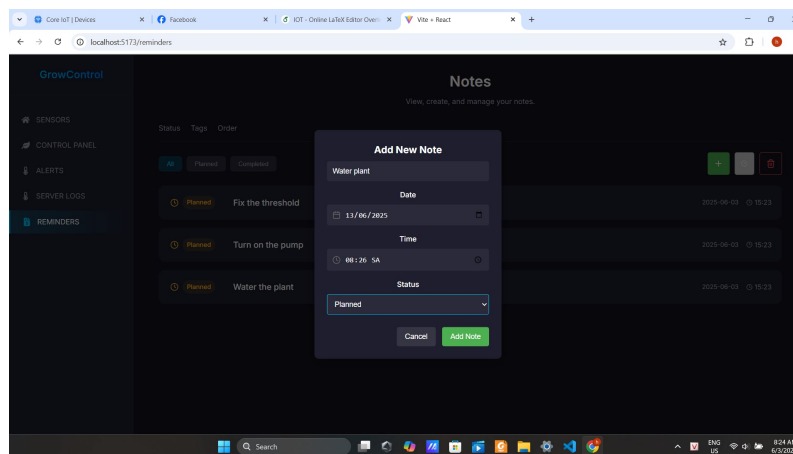


Figure 44: Add Notes interface for scheduling tasks with date, time, and status fields

14.2.5 Create Reminder

The GrowControl system enhances task management through its **Reminders** section, specifically with the *Create Reminder* functionality, allowing users to schedule and track essential activities for maintaining optimal environmental conditions. This feature is accessible via the Notes

tab, where users can view, create, and manage reminders with ease.

To create a reminder, users can click the green “Add New Note” button, which opens a modal window for inputting task details. The interface displays existing reminders, such as “Fix the threshold,” “Turn on the pump,” and “Water the plant,” all scheduled for 13/06/2025 at specific times like 08:26 or 15:23. Each reminder entry includes a status indicator, with all current examples marked as “Planned,” signifying tasks that are scheduled but not yet completed. This status can be updated to “Completed” once the task is executed, providing a clear overview of pending and finished activities.

The Notes section also offers filtering options, with tabs labeled “All,” “Planned,” and “Completed,” enabling users to sort reminders based on their status. Additional icons for adding, editing, or deleting reminders are present, with the delete option highlighted by a red trash bin icon, allowing users to remove outdated or irrelevant tasks. This functionality ensures that the reminder list remains organized and relevant, supporting efficient task management within the GrowControl system.

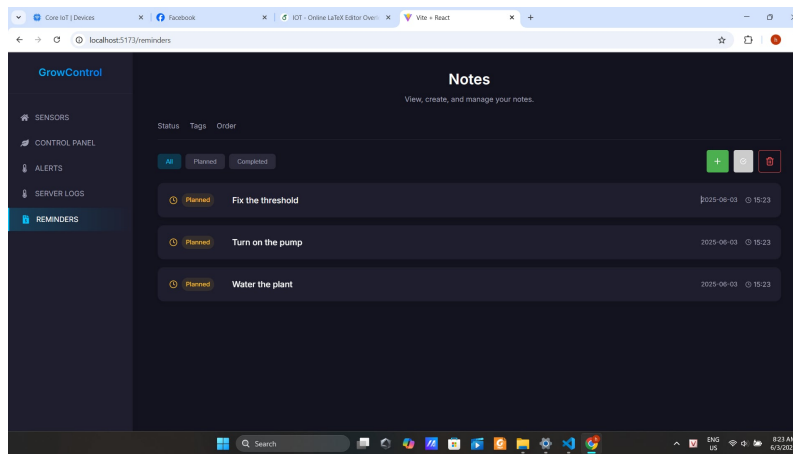


Figure 45: Create Reminder interface displaying scheduled tasks with status and time details

14.2.6 Configure Status of Reminder

The GrowControl system’s **Reminders** section allows users to efficiently manage tasks by configuring the status of reminders, ensuring accurate tracking of completed and pending activities. This functionality is crucial for maintaining an organized workflow in a controlled agricultural environment.

In the Notes interface, reminders such as “Fix the threshold,” “Turn on the pump,” and “Water the plant” are listed with their respective statuses, dates, and times. Initially, these tasks may be marked as “Planned,” indicating they are scheduled for future execution, as seen with the latter two tasks set for 13/06/2025 at 15:23. However, once a task is completed, users can update its status to reflect this change. For example, the task “Fix the threshold” has been marked as “Completed,” indicated by a green checkmark and the label “Completed,” showing that the task was finished on 13/06/2025 at 15:23.

To configure the status, users can edit a reminder by selecting the edit icon (a gray pencil), which opens a form similar to the one used for adding new notes. Here, they can change the status from “Planned” to “Completed” and save the update. This action updates the reminder’s visual indicator in the list, providing a clear distinction between ongoing and finished tasks. The interface also supports filtering options like “All,” “Planned,” and “Completed,” allowing users to focus on specific task categories. Additionally, the delete option, marked by a red trash bin icon, enables users to remove completed or irrelevant reminders, keeping the list streamlined.

This status configuration feature enhances task management by providing a clear overview of task progress, ensuring that users can prioritize their efforts and maintain optimal environmental conditions effectively.

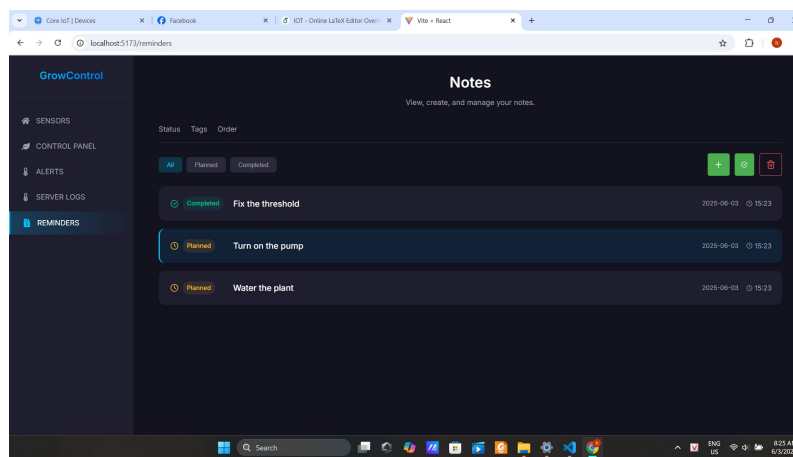


Figure 46: Configure Status of Reminder interface showing completed and planned tasks

14.2.7 Notification Server Log

The GrowControl system’s **Notification Server Log** provides a comprehensive record of system activities, including device adjustments, threshold value modifications, and user login events. This feature, accessible under the Server Logs section, ensures that all significant actions are logged with detailed timestamps and statuses, enabling users to monitor and audit system operations effectively.

The notification log captures a variety of events, each marked with a status to indicate its completion. For instance, notifications related to device adjustments are prominently featured, such as the “Started Light in On mode in Sector A,” logged on June 3, 2025, and marked as “Completed.” This indicates that the light device in Sector A was successfully activated, reflecting real-time control actions initiated via the Control Panel. Similarly, threshold value changes are meticulously recorded, including “Set threshold for Light to 500 lux in Sector A,” “Set threshold for Humidity to 60

Additionally, the log includes notifications for user interactions, such as “User logged in,” which appears multiple times on June 3, 2025, each with a “Completed” status. This tracking of login events enhances security and accountability, allowing administrators to monitor access patterns and detect any unauthorized attempts. Each notification entry includes a timestamp, status (e.g.,

“Completed”), and metadata like upvotes and comments (currently zero), providing a structured overview of system activity.

Users can filter notifications using options such as “All,” “Pending,” “Reviewing,” “Planned,” “In Progress,” “Completed,” and “Closed,” offering flexibility in reviewing the log based on task progress. The log’s design supports historical analysis, enabling users to correlate device adjustments and threshold changes with environmental outcomes, while the inclusion of login notifications aids in maintaining system integrity. This robust logging mechanism is essential for troubleshooting, optimizing system performance, and ensuring reliable operation of the GrowControl environment.

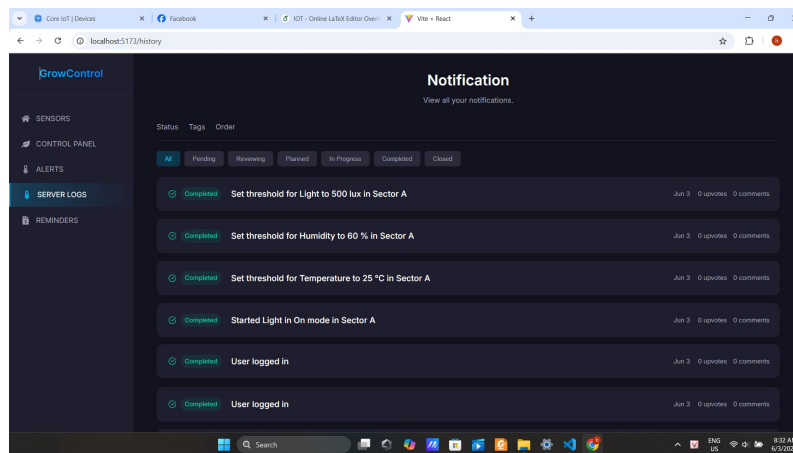


Figure 47: Notification Server Log interface showing device adjustments, threshold changes, and login events

14.2.8 Offline Mode

The GrowControl system is designed to operate seamlessly in various network conditions, including scenarios where connectivity is lost, entering what is referred to as **Offline Mode**. This subsection examines the system’s behavior in offline mode, focusing on its impact on sensor data visualization and user interaction.

In offline mode, the system cannot retrieve real-time data from sensors, as evidenced by the Sensors section for Sensor 02 in Sector A. The interface displays “N/A°C” for temperature, “N/A%” for humidity, and “N/A lux” for light, indicating that no current sensor readings are available due to the lack of network connectivity. The Graphical Synopsis section, which typically provides visual representations of temperature, humidity, and light trends over time (with options like Hour, Day, Week, Month, Year, and Custom), remains blank in this state, as no data can be fetched or updated.

Despite the absence of live data, the interface retains its navigational functionality, allowing users to access other sections like Control Panel, Alerts, Server Logs, and Reminders. This ensures that users can still review historical data, manage reminders, or configure settings that do not require real-time sensor input. The system is likely designed to resume normal operation once connectivity is restored, automatically syncing any locally stored data and updating the sensor

readings accordingly. Offline Mode highlights the system’s resilience, enabling users to maintain operational awareness and perform non-data-dependent tasks even during network disruptions.

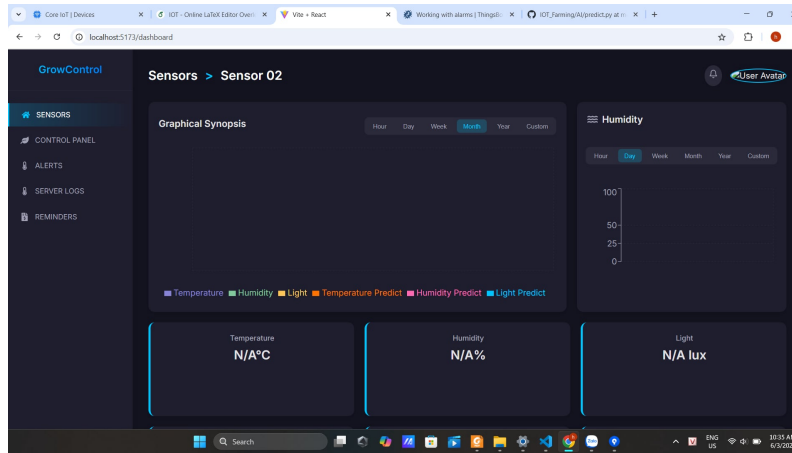


Figure 48: Offline Mode interface showing unavailable sensor data

14.2.9 Data Receiving Mode

The GrowControl system operates in a **Data Receiving Mode** when it is online and actively collecting data from sensors, as demonstrated in the interface for Sensor 02. This subsection details how the system receives, processes, and visualizes real-time environmental data, including current values, historical trends, and predictive analytics, ensuring users have a comprehensive view of the controlled environment.

In this mode, the system retrieves and displays real-time data from Sensor 02, covering temperature, humidity, and light intensity. The current readings are presented prominently: temperature at 30.5°C with a 2.3

The **Graphical Synopsis** section enhances data interpretation by offering visual representations of these metrics over selectable time frames, including Hour, Day, Week, Month, Year, and Custom, with the current view set to Month. The bar chart displays historical data with distinct colors for each parameter: purple for temperature, green for humidity, yellow for light, orange for temperature prediction, pink for humidity prediction, and blue for light prediction. For example, the chart shows varying levels of these parameters over the month, with peaks and troughs indicating fluctuations that users can analyze to adjust settings.

Additionally, the Humidity panel provides a dedicated line graph, set to a Day view, showing a stable value around 72.0

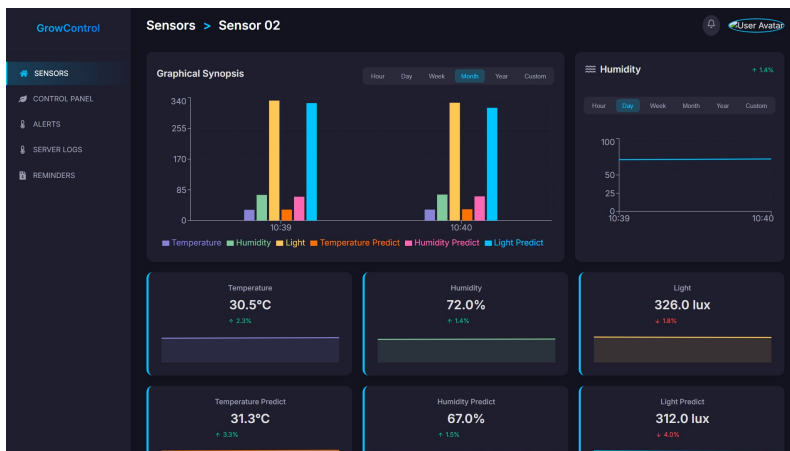


Figure 49: Data Receiving Mode interface showing real-time sensor data and graphical synopsis

14.2.10 Summary of GrowControl Website

The GrowControl website provides a user-friendly IoT platform for managing agricultural environments. The **Control Panel** offers real-time device control (lights, fans, pumps) through an intuitive interface. **Threshold Settings** allow configuration of environmental parameters (temperature, humidity, light) with current value comparisons. The **Alerts** and **Notification Server Log** sections display threshold breaches and system events like logins for monitoring. **Reminders** enable task scheduling and status updates (Planned/Completed) via the Notes section. In **Offline Mode**, the interface shows "N/A" for sensor data, while **Data Receiving Mode** delivers real-time data (e.g., 30.5°C, 72.0% humidity) and predictive trends through the **Graphical Synopsis**. This website ensures efficient environmental control and task management.

15 Summary

15.1 Perspective

Project Summary: IoT Smart Farming System

Reflection on the Initial Plan: As this was our first time approaching the design and implementation of an IoT-based Smart Farming system, it is understandable that some shortcomings and errors occurred during development. Nevertheless, the project has made notable progress, which can be summarized as follows:

Achievements:

- Successfully built a basic IoT system with essential functions such as data reading, device control, and value processing.
- In terms of AI capabilities, the system is able to detect fire incidents and diseased yellow leaves, and can predict future sensor values based on current readings.
- Gained a solid understanding of how data transmission and communication operate between different components of the system.

Unachieved Goals:

- Advanced functionalities still contain many bugs and remain incomplete, such as:
- Generating parameter adjustment recommendations based on current sensor readings and the type of crops being grown.
- High latency in updating real-time environmental sensor values.
- The software has not been fully optimized in terms of performance and user experience.

15.2 Function Extension

As agriculture moves closer to smarter and more autonomous technology, the YOLO Farm System strives to improve on a continuous basis in order to meet the increasing demands of modern agriculture. Future enhancements are proposed to improve functionality and convenience of use. Upgrades enhance the system's flexibility, accuracy, scalability, and ease of use, resulting in a smarter, more efficient, and integrated farm management system.

- **Multi-Account Management:** Manage multiple user accounts with varying responsibilities and permissions for farm workers, managers, and administrators.
- **Device Usage Analytics:** Analyze farm device usage to improve efficiency and maintenance schedules.

- **Improved Plant Disease Detection:** Expand the YOLO model to identify and categorize more diseases in dragon fruit plants, leading to better diagnostic accuracy and farm management.
- **Integration of Additional Devices:** Connect new devices, such as alarm systems, to provide real-time emergency notifications, hence boosting farm security and response.
- **Expand the camera network:** For better coverage, accuracy, and scalable monitoring.
- **Automated Monthly Reporting:** Users can receive complete reports on system performance, crop health, and device usage via email.
- **Multilingual Support:** Add language switching to the system interface for global accessibility.

15.3 Conclusion

This project helps our team gain a deeper understanding of how to design and implement AIoT-driven systems to increase productivity and product quality in agriculture. By combining a network of environmental sensors (e.g. temperature, humidity, soil moisture, light) with image-based disease detection, the platform continuously monitors crop health and automatically adjusts key parameters, such as irrigation, lighting, or ventilation - to reduce manual labor for farmers and maintain optimal growing conditions. It also detects fires based on images and takes timely action.

In essence, the system leverages artificial intelligence to help farmers quickly and accurately identify common diseases or signs of stress in plants. Whenever the model detects early-stage symptoms—such as yellow leaves, fungal spots, or insect infestation—the platform immediately issues an alert (via a mobile/web dashboard and optional SMS text) along with recommended corrective actions. In doing so, farmers can intervene earlier, apply targeted treatments, and prevent the spread of disease.

In short, the project is a practical step toward smart farming by integrating real-time sensor feedback, AI-driven image analysis, and autonomous control. As we continue to refine the platform—improving model accuracy, expanding crop coverage, and streamlining hardware deployment—we believe the platform will evolve into a fully scalable solution that empowers modern farmers with actionable insights, greater efficiency, and higher-quality yields.



16 Link to our Implementation

Here are our work for the project:

- [IOT_Farming GitHub Repository](#)
- [Other Documents Submission](#)
- [AI Detection Github Repository \(WebApp_IOT\)](#)

References

- [1] Final I Fire Detection (Amenazas) Dataset. (n.d.). Roboflow Universe. Retrieved June 3, 2025, from <https://universe.roboflow.com/finalifiredetection/amenazas-ebbn9>
- [2] OHstem. (n.d.). OHstem Search Results. Retrieved June 3, 2025, from <https://ohstem.vn/?srsltid=AfmB0ord2zTfFa4XuhAEdfuYs-rTC88epGHrCATxGw12o4WjkvjJJmiy>
- [3] ThingsBoard. (n.d.). Rule Chains — ThingsBoard User Guide. Retrieved June 3, 2025, from <https://thingsboard.io/docs/user-guide/ui/rule-chains/>
- [4] ThingsBoard. (n.d.). Entities Aliases — ThingsBoard User Guide. Retrieved June 3, 2025, from <https://thingsboard.io/docs/user-guide/ui/aliases/>
- [5] Intel Vietnam. (n.d.). Edge Computing & Edge Devices. Retrieved June 3, 2025, from <https://www.intel.vn/content/www/vn/vi/edge-computing/edge-devices.html>
- [6] ThingsBoard. (n.d.). Device Profiles — ThingsBoard User Guide. Retrieved June 3, 2025, from <https://thingsboard.io/docs/user-guide/device-profiles/>
- [7] How2Electronics. (n.d.). ESP32-CAM Based Object Detection & Identification with OpenCV. Retrieved June 3, 2025, from <https://how2electronics.com/esp32-cam-based-object-detection-identification-with-opencv/>
- [8] CoreIoT. (n.d.). CoreIoT — IoT Solutions & Platform. Retrieved June 3, 2025, from <https://coreiot.io/>
- [9] Ultralytics. (n.d.). *Ultralytics YOLO GitHub repository*. GitHub. Retrieved April 29, 2025, from <https://github.com/ultralytics/ultralytics>
- [10] Al-Naji, A., Chahl, J., & Mohamed, A. (2024). AIoT applications in smart agriculture: A comprehensive review. *Journal of Network and Computer Applications*, **225**, 103775. <https://doi.org/10.1016/j.jnca.2024.103775>
- [11] Kumar, R., & Lee, B. (2023). Challenges of AIoT in smart agriculture: A review. *Sensors*, **23**(7), 3752. <https://www.mdpi.com/1424-8220/23/7/3752>
- [12] Espressif Systems. (2020). *ESP32 Series Datasheet*. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [13] OASIS. (2014). *MQTT Version 3.1.1 Specification*. Retrieved from <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
- [14] npm, Inc. (n.d.). *Docs—npm Documentation*. Retrieved June 3, 2025, from <https://docs.npmjs.com/>



- [15] Nguyen Thi Thuy Linh. (n.d.). Xay dung JSON Server voi thu vien JSON-Server. *Viblo*. Retrieved June 3, 2025, from <https://viblo.asia/p/xay-dung-json-server-voi-thu-vien-json-server-07LKX7045V4>